

UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Scienze Economiche e Statistiche

Corso di Laurea Magistrale in Scienze Statistiche per la Finanza

Prova finale in
Analisi delle Serie Storiche

LE EXTREME LEARNING MACHINES PER SERIE STORICHE NON LINEARI: UN'ANALISI COMPARATIVA CON LE RETI NEURALI ARTIFICIALI

Relatore:
Ch.ma Prof.ssa
Cira Perna

Candidato:
Simone Squizzato
matr. 0222401031

ANNO ACCADEMICO 2025/2026

Indice

Introduzione **4**

1 Reti Neurali: definizione e descrizione **5**
1.1 Il neurone come unità di elaborazione 5
1.2 Un modello generale per il singolo neurone 6
1.3 La connessione tra più neuroni: reti neurali feedforward 6
1.3.1 Reti neurali shallow 7
1.3.2 Reti neurali deep 8
1.4 La connessione tra più neuroni: reti neurali ricorrenti 8
1.4.1 Reti di Elman 9
1.4.2 Reti di Jordan 9
1.4.3 Reti LSTM 10
1.5 Il ruolo delle funzioni di attivazione per la nonlinearità 10
1.5.1 Funzione di attivazione logistica 11
1.5.2 Funzione tangente iperbolica (tanh) 11
1.5.3 Funzione ReLU e Leaky ReLU 12
1.5.4 Funzione ELU 13
1.5.5 Funzione Swish e Hard-Swish 14
1.6 Capacità di approssimazione di funzioni non lineari 15
1.7 L'addestramento della rete come problema di ottimizzazione 16
1.7.1 Il ruolo degli algoritmi di ottimizzazione 16
1.7.2 La tecnica della backpropagation per la computazione del gradiente 20

2 Le reti neurali nel contesto delle serie storiche non lineari **24**
2.1 Modelli per serie storiche non lineari 24
2.2 Modelli di reti neurali per serie storiche non lineari 25
2.2.1 Previsioni con reti neurali non lineari autoregressive 26
2.3 Cross-validation per serie storiche 26
2.3.1 Misure di accuratezza previsiva 27

3 Le Extreme Learning Machines **29**
3.1 Le Extreme Learning Machines per la stima delle reti neurali artificiali . . . 29
3.2 Previsione delle serie storiche con le Extreme Learning Machine 32

4 Caso studio: analisi comparativa dei tempi di addestramento con dati simulati **38**
4.1 Modelli e metodologia utilizzati 38
4.1.1 Modello EXPAR 38
4.1.2 Impostazione del disegno sperimentale 39
4.2 Risultati 40
4.2.1 Reti neurali stimate su un processo EXPAR(1) 40
4.2.2 Reti neurali stimate su un processo EXPAR(2) 43
4.3 Conclusioni e sviluppi futuri 46

Elenco delle figure

1.1 Neurone di McCulloch-Pitts 5
1.2 Modello generale per il singolo neurone 6
1.3 Grafo di una rete neurale feedforward con un solo strato nascosto 7
1.4 Grafo di una rete neurale deep feedforward 8
1.5 Rete neurale ricorrente di Elman 9
1.6 Rete neurale ricorrente di Jordan 10
1.7 Funzione logistica 11
1.8 Funzione tangente iperbolica (tanh) 12
1.9 Funzione ReLU e sua derivata 13
1.10 Funzione Leaky ReLU e sua derivata 14
1.11 Funzione ELU e sua derivata 14
1.12 Funzione Hard-Swish 15
1.13 Tasso di apprendimento grande vs piccolo 18
1.14 Comparazione del processo di apprendimento 18

2.1 Rete neurale autoregressiva di ordine p, NNAR(p,r) 25
2.2 Time Series Cross-Validation 26

3.1 Rappresentazione della topologia di una Extreme Learning Machine (ELM) 30
3.2 Rappresentazione della topologia di una rete ELM-NAR 34
3.3 Rappresentazione della topologia di una rete ELM-NAR con previsione diretta a h step avanti 35
3.4 Rappresentazione del grafo di una rete ELM-NAR multi-input multi-output per la previsione multi-step 36

4.1 Rappresentazione grafica di una realizzazione di una serie simulata da un processo EXPAR(2) con numerosità campionaria $T = 1000$. 39

Elenco delle tabelle

4.1 Confronto delle prestazioni in-sample (RMSE e rapporto dei tempi di addestramento) tra reti neurali feedforward (NNET) ed Extreme Learning Machines (ELM) su dati simulati da un processo EXPAR(1), al variare della numerosità campionaria (T) e dei neuroni nello strato nascosto (m) 41

4.2 Rapporto tra i tempi di addestramento in-sample a parità di capacità di adattamento (tolleranza RMSE del 5%) tra NNET ed ELM per il modello EXPAR(1). 42

4.3 Prestazioni predittive out-of-sample e confronto dei tempi di addestramento tra NNET ed ELM (con m^* neuroni) a parità di errore di addestramento (tolleranza 5%) per il processo EXPAR(1). 42

4.4 Confronto delle prestazioni in-sample (RMSE e rapporto dei tempi di addestramento) tra reti neurali feedforward (NNET) ed Extreme Learning Machines (ELM) su dati simulati da un processo EXPAR(2), al variare della numerosità campionaria (T) e della complessità strutturale (m). 44

4.5 Rapporto tra i tempi di addestramento in-sample a parità di capacità di adattamento (tolleranza RMSE del 5%) tra NNET ed ELM per il modello EXPAR(2). 44

4.6 Prestazioni predittive out-of-sample e confronto dei tempi di addestramento tra NNET ed ELM (con m^* neuroni) a parità di errore di addestramento (tolleranza 5%) per il processo EXPAR(2). 45

Introduzione

L'analisi e la previsione delle serie storiche rappresentano uno dei campi di ricerca più attivi e stimolanti nell'ambito della statistica computazionale e dell'econometria. Nel settore economico e finanziario, le serie osservate esibiscono frequentemente dinamiche complesse caratterizzate da *jump phenomena*, volatilità asimmetrica e regimi multipli; in tali scenari, i modelli lineari tradizionali, pur mantenendo un'intrinseca eleganza e interpretabilità, risultano spesso inadeguati nel catturare la vera natura del processo stocastico sottostante. Per tale ragione, negli ultimi decenni l'attenzione si è spostata in modo deciso verso modelli non lineari e tecniche di Machine Learning.

Tra gli strumenti più potenti e flessibili vi sono le Reti Neurali Artificiali (ANN), che, grazie ai teoremi di approssimazione universale, non richiedono la specificazione a priori dell'esatta forma funzionale del meccanismo generatore dei dati, offrendo uno strumento data-driven di straordinaria efficacia. Tuttavia, l'addestramento dei classici modelli non lineari autoregressivi basati su reti neurali (NNAR) tramite l'algoritmo di *backpropagation* solleva importanti sfide: risulta computazionalmente dispendioso, sensibile all'intrappolamento in minimi locali della funzione di costo e vulnerabile al problema dell'overfitting, specialmente quando la dimensione del campione è esigua rispetto alla complessità (numero di regressori e nodi nascosti) della rete.

Il presente lavoro di tesi si propone di analizzare una metodologia di stima alternativa e altamente efficiente: le *Extreme Learning Machines* (ELM). Questo approccio si basa sull'idea innovativa di determinare casualmente i parametri dello strato nascosto, riducendo l'addestramento della rete alla semplice soluzione analitica di un problema di regressione lineare regolarizzata per i pesi di output. Il core del lavoro consiste in un'analisi comparativa, strutturata su dati simulati, tesa a valutare quando e in che misura le ELM rappresentino un'alternativa superiore o complementare rispetto all'addestramento classico.

L'elaborato è strutturato in quattro capitoli. Il primo capitolo introduce i fondamenti teorici delle Reti Neurali Artificiali, descrivendo l'unità elaborativa di base (il neurone), le principali topologie (architetture feedforward e ricorrenti) e il ruolo cruciale delle funzioni di attivazione nell'indurre la non linearità. Viene inoltre passato in rassegna il problema di ottimizzazione alla base dell'apprendimento, con un focus sulla discesa del gradiente e sulla tecnica di *backpropagation*.

Il secondo capitolo inserisce le reti neurali nel contesto specifico dell'analisi delle serie storiche, trattando il passaggio dai tradizionali modelli autoregressivi lineari e non lineari alle reti NNAR, affrontando inoltre le tematiche della *Time Series Cross-Validation* e delineando le principali metriche di accuratezza impiegate per la valutazione dell'errore predittivo.

Il terzo capitolo è dedicato alla teoria delle *Extreme Learning Machines*. Si analizza la formulazione matematica del modello e l'inversione generalizzata di Moore-Penrose; la discussione viene quindi specializzata per l'adattamento delle ELM alla previsione di serie storiche (ELM-NAR), illustrando sia gli approcci iterativi che quelli multi-step diretti.

Infine, il quarto capitolo costituisce il nucleo empirico della tesi. Tramite l'implementazione di uno studio di simulazione basato su processi *Exponential Autoregressive* (EXPAR(1) ed EXPAR(2)), i modelli ELM e NNET vengono confrontati lungo tre dimensioni: capacità di adattamento in-sample, accuratezza predittiva out-of-sample e tempi di addestramento. Vengono testate diverse configurazioni di numerosità campionaria e di dimensionalità dell'input, allo scopo di fornire interpretazioni statistiche sul trade-off tra il costo computazionale e l'accuratezza previsiva, le quali troveranno sintesi nella sezione conclusiva.

1. Reti Neurali: definizione e descrizione

1.1 Il neurone come unità di elaborazione

Le reti neurali artificiali sono modelli computazionali e statistici ispirati al sistema nervoso degli esseri viventi. Inizialmente sviluppate da scienziati cognitivi interessati a comprendere il funzionamento del cervello nei processi computazionali, le reti neurali artificiali (ANN) rappresentano una classe di modelli non lineari; sebbene traggano ispirazione dal modo in cui il cervello elabora le informazioni, questi modelli sono ancora molto lontani dal fornire una rappresentazione realistica di come questo effettivamente funzioni; tuttavia, offrono una grande flessibilità, che li rende utili in numerosi settori e in numerose applicazioni.

L'unità di elaborazione di base in una rete neurale è il neurone; si tratta di un nodo che riceve tutti gli input provenienti da altri nodi e genera un output secondo una funzione di trasferimento chiamata funzione di attivazione. La prima formulazione matematica di un neurone somigliante ai corrispettivi biologici è stata proposta nel 1943 da Warren McCulloch (neurofisiologo e cibernetico statunitense) e Walter Pitts (logico che ha lavorato nel campo delle neuroscienze computazionali).

In questa prima modellazione, ogni input può assumere solo due stati: 0 (non attivato) o 1 (attivato); quale dei due prevalga dipende dal valore della funzione di attivazione, ossia una funzione a gradino che può assumere solo due valori (0 e 1), mentre θ rappresenta un livello di soglia che determina quando l'output y viene attivato.

Matematicamente:

$$y = H(\sum_{j=1}^{d} i_j - \theta) \tag{1.1}$$

dove $i_j \in \{0, 1\}$, con $j = 1, 2, \ldots, d$ e H è la Heaviside step function (anche nota come funzione *hardlim*) definita come segue:

$$H(z) = \begin{cases} 0, & se \quad z \leq 0 \\ 1, & se \quad z > 0 \end{cases} \tag{1.2}$$

dalla definizione di questa funzione risulta chiaro come l'unico parametro che regola l'attivazione e l'assegnazione dei valori della funzione sia θ, che è anche l'unico parametro da

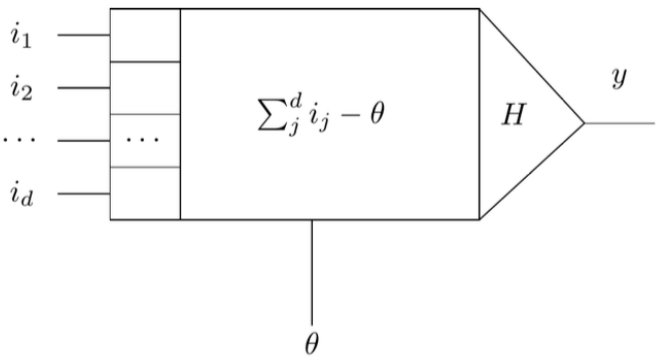


Figura 1.1: Neurone di McCulloch-Pitts

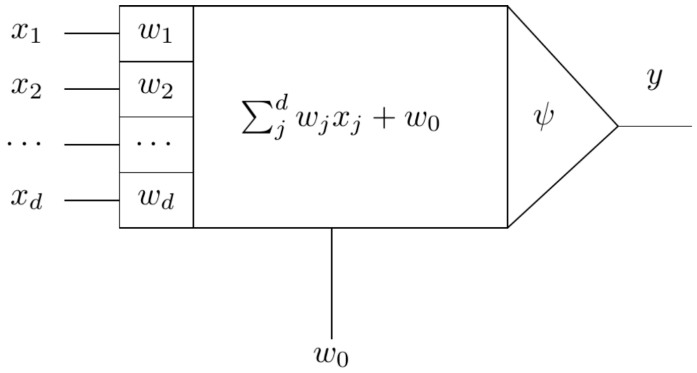


Figura 1.2: Modello generale per il singolo neurone

stimare nei dati. Per fare ciò, sarà necessario introdurre una funzione di perdita e scegliere θ in modo da minimizzare tale funzione; un esempio di funzione di perdita è la funzione quadratica, definita nel modo seguente:

$$Loss(\theta) = \sum_{k=1}^{n} [y_k - \hat{y}_k(x_k, \theta)]^2 \tag{1.3}$$

dove y_k è il valore osservato, mentre $\hat{y}_k(x_k, \theta)$ è il valore predetto dal modello in corrispondenza dell'insieme di variabili osservate x_k e della scelta del parametro θ.

1.2 Un modello generale per il singolo neurone

Per poter generalizzare il modello per singolo neurone sviluppato da McCulloch e Pitts, oltre ad avere un output y, supponiamo di avere x_d input (possibilmente con valori reali), a ciascuno dei quali è associato un peso w_j e una funzione di attivazione $\psi(\cdot)$ (non necessariamente Heaviside). In questo contesto, il valore della soglia, originariamente indicato con θ, verrà sostituito da w_0. Il modello così descritto permette di identificare il cosiddetto percettrone e può essere rappresentato graficamente come nella Figura 1.2; matematicamente, invece, è definito come segue:

$$y = \psi \left(\sum_{j=1}^{d} w_j x_j + w_0 \right) \tag{1.4}$$

dove x_j è il j-esimo input, w_j è il peso associato al j-esimo input e w_0 è il bias (o la soglia).

Come anticipato, la funzione ψ è chiamata funzione di attivazione; il suo ruolo principale è trasformare la somma pesata degli input ricevuti dal nodo precedente in un valore di output, che identificherà l'output della rete; per questo motivo, la funzione di attivazione è solitamente scelta continua, a valori reali e sigmoidale in modo tale da restituire valori nell'intervallo [0,1], ossia:

$$\psi : \mathbb{R} \to [0, 1] \tag{1.5}$$

Lo scopo di tale funzione è aggiungere non linearità alla rete neurale, per cui viene definita attraverso l'utilizzo di funzioni non lineari (continue o discontinue).

1.3 La connessione tra più neuroni: reti neurali feedforward

Si inizia a discutere del concetto di rete neurale nel momento in cui due o più neuroni (o percettroni) vengono collegati tra loro per produrre un output che diventerà l'input di un

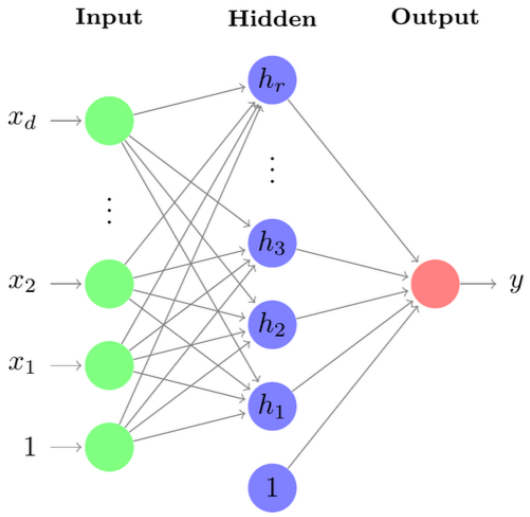


Figura 1.3: Grafo di una rete neurale feedforward con un solo strato nascosto

differente neurone; tali collegamenti possono avvenire sulla base di infinite combinazioni di layout che identificano gli strati (o layer) e le caratteristiche della rete stessa.

1.3.1 Reti neurali shallow

Nelle reti neurali artificiali, i neuroni vengono quindi organizzati in strati, che permettono di rappresentare la topologia di una rete attraverso un grafo, come mostrato nella Figura 1.3. In particolare, la Figura riporta la rappresentazione di una rete neurale formata da:

- Un layer di input composto da d neuroni;
- Un *hidden layer* composto da r neuroni (tipico delle reti shallow);
- Un layer di output formato da un singolo neurone.

La struttura della rete appena descritta identifica una rete neurale artificiale feedforward, denominata in questo modo perchè le informazioni transitano dai neuroni di input ai neuroni di output in una sola direzione (in "avanti"), al contempo questa rete è anche detta *shallow* in quanto composta da un singolo strato nascosto a differenza delle reti *deep* che sono composte da un numero superiore di strati nascosti. Le reti neurali *shallow* sono solitamente identificate con la dicitura $NN(d, r)$, dove d e r indicano la grandezza dello strato di input e dello strato nascosto, rispettivamente.

Dal punto di vista matematico, una rete neurale feedforward, $NN(d, r)$, è definita dalla seguente espressione:

$$y = \phi\left(\sum_{k=1}^{r} c_k h_k + c_0\right) \tag{1.6}$$

dove:

$$h_k = \psi\left(\sum_{j=1}^{d} w_{kj} x_j + w_{k0}\right) \tag{1.7}$$

e dove $\psi(\cdot)$ è la funzione di attivazione dei neuroni nello strato nascosto, mentre $\phi(\cdot)$ è la funzione di attivazione del neurone di output.

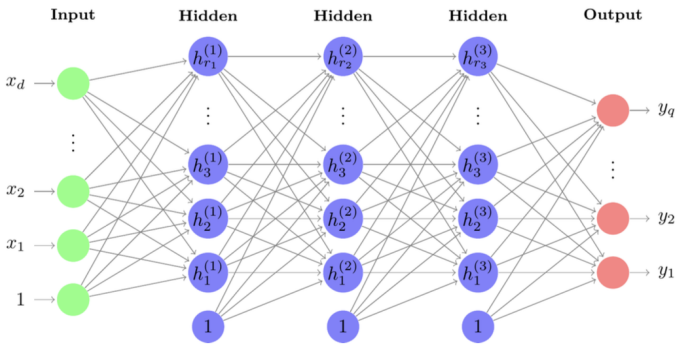


Figura 1.4: Grafo di una rete neurale deep feedforward

In alcuni casi, le reti neurali feedforward vengono anche riconosciute con la nomenclatura di *multi-layer perceptrons* (MLP); tuttavia, in altri casi, il termine MLP è usato per riferirsi a uno specifico tipo di rete costituita da specifiche funzioni di attivazione o addestrata con uno specifico algoritmo.

1.3.2 Reti neurali deep

Le reti neurali feedforward vengono definite *deep* (o profonde) quando il numero di strati nascosti (*hidden layer*) è pari o superiore a due, oppure in presenza di un numero di neuroni di output maggiore di uno. L'architettura di questo tipo di rete feedforward è definita da due iperparametri:

1. Il numero di strati;
2. Il numero di nodi in ciascuno strato.

Il modello matematico alla base delle reti neurali profonde è molto più complesso rispetto alle reti *shallow*; in particolare, è necessario definire, per l'appunto, il numero di strati (inclusi i layer di input e output), denotati da r^{net} e il numero di nodi dell'l-esimo strato, denotato come r_l^{net}.

Matematicamente, l'output del k-esimo neurone del primo strato, denotato da $y_{k,1}$, è calcolato come segue:

$$y_{k,1} = \psi_{k1}\left(\sum_{j=1}^{d} w_{kj}^{1} x_d + w_{k0}^{1}\right), \quad k = 1, 2, \ldots, r_1^{net} \tag{1.8}$$

Mentre l'output del k-esimo neurone nello strato nascosto e nello strato di output l, denotato da $y_{k,l}$, è calcolato come segue:

$$y_{k,l} = \psi_{kl}\left(\sum_{j=1}^{r_{l-1}^{net}} w_{kj}^{l} y_{j,l-1} + w_{k0}^{l}\right), \quad k = 1, 2, \ldots, r_l^{net} \quad \text{e} \quad l = 1, 2, \ldots, r^{net} \tag{1.9}$$

dove w_{kj}^{l} sono i pesi associati al j-esimo segnale di input ricevuto dal neurone (k, l).

1.4 La connessione tra più neuroni: reti neurali ricorrenti

Le reti neurali ricorrenti (Recurrent Neural Networks, RNNs) sono caratterizzate da un'architettura dove le informazioni e gli output dei neuroni fluiscono in modo circolare; per cui

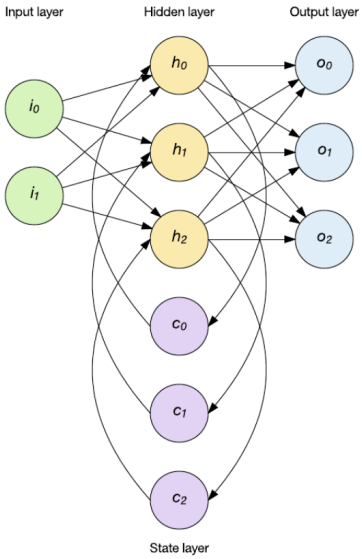


Figura 1.5: Rete neurale ricorrente di Elman

risultano particolarmente adatte a trattare dati che presentano dipendenze temporali. In questa tipologia di rete esiste almeno una connessione di feedback in cui l'output di un neurone può essere reintrodotto come input per lo stesso neurone o per un altro neurone, creando un ciclo. Negli anni '80 sono state introdotte due principali tipologie di reti neurali ricorrenti: le reti di Elman e le reti di Jordan.

1.4.1 Reti di Elman

Una rete neurale ricorrente di Elman è una rete neurale a tre strati che utilizza un insieme di unità di contesto, tali unità sono neuroni presenti in quello che viene definito *state layer* che ricevono informazioni dallo strato nascosto, come mostrato nella Figura 1.5. Queste informazioni successivamente vengono utilizzate per influenzare le attivazioni dello stesso layer nascosto nelle iterazioni successive. Dal punto di vista matematico la rete viene definita come segue:

$$y_t = \phi \left(\sum_{j=1}^{r} c_j h_{jt} + c_0 \right) \tag{1.10}$$

dove:

$$h_{jt} = \psi(\mathbf{x}_t^T \mathbf{w}_j + a_j h_{j,t-1} + w_0) \tag{1.11}$$

1.4.2 Reti di Jordan

Una rete neurale ricorrente di Jordan è simile nella struttura a una rete di Elman. La principale differenza tra le due tipologie risiede nel modo in cui sono alimentate le unità di contesto, infatti, nelle reti di Jordan è il layer di output che influenza lo stato delle unità di contesto e non il layer nascosto, come mostrato nella Figura 1.6. Una volta aggiornato, lo state layer influenza lo strato nascosto modificando l'output delle iterazioni successive.

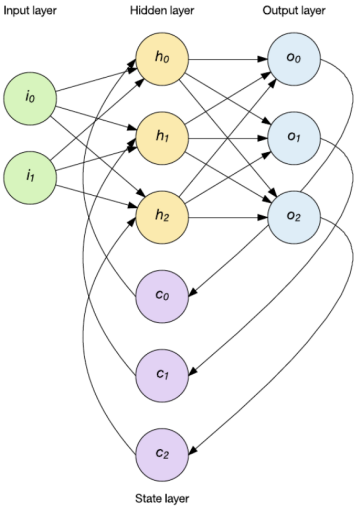


Figura 1.6: Rete neurale ricorrente di Jordan

Formalmente, le reti di Jordan sono definite come segue:

$$y_t = \phi\left(\sum_{j=1}^{r} c_j h_{jt} + c_0\right) \tag{1.12}$$

dove, in questo caso, avremo che:

$$h_{jt} = \psi(\mathbf{x}_t^T \mathbf{w}_j + \mathbf{a}_j \mathbf{y}_{t-1}^T + w_0) \tag{1.13}$$

1.4.3 Reti LSTM

La terza e ultima tipologia di reti neurali ricorrenti sono le reti LSTM (Long Short Term Memory) che sono state istituite per risolvere il problema del *vanishing gradient* (scomparsa del gradiente) e per gestire i dati sequenziali di lunga durata. Per farlo si introducono nella struttura della rete delle celle di memoria che regolano il flusso di informazioni attraverso porte (input, forget e output gate), consentendo di "dimenticare" informazioni irrilevanti e "ricordare" quelle importanti.

1.5 Il ruolo delle funzioni di attivazione per la nonlinearità

Come anticipato, lo scopo dell'utilizzo di funzioni di attivazione che mappano l'asse reale nell'intervallo [0,1] è introdurre non linearità nella rete neurale. Il passaggio degli input attraverso queste funzioni negli strati nascosti è necessario a evitare che la struttura della rete collassi in una semplice regressione lineare; infatti, è possibile dimostrare quanto detto mediante un banale esempio. Supponiamo di avere una rete neurale composta da un singolo neurone di input, un singolo strato nascosto composto anch'esso da un solo neurone e un singolo neurone di output in cui la funzione di attivazione dello strato nascosto e dell'output è la funzione identità ($\psi(z) = \phi(z) = z$); in questo contesto, l'output della rete neurale sarà

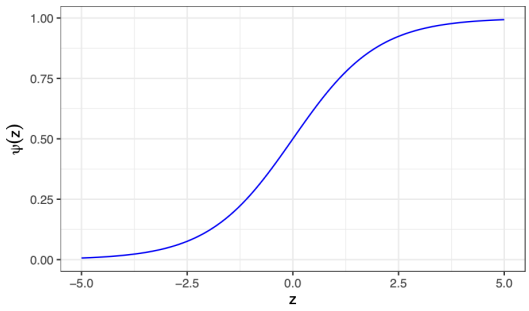


Figura 1.7: Funzione logistica

calcolato come:

$$\begin{aligned} y &= \phi(c_1\psi(w_1x + w_0) + c_0) \\ &= \phi(c_1(w_1x + w_0) + c_0) \\ &= c_1w_1x + c_1w_0 + c_0 \\ &= \beta_1x + \beta_0 \end{aligned} \tag{1.14}$$

per cui le funzioni di attivazione così definite trasformano la rete in un singolo strato che si comporta come una combinazione lineare degli input.

Per quanto detto e dimostrato, risulta fondamentale conoscere le diverse funzioni di attivazione, con le loro proprietà e i loro limiti; di seguito passeremo in rassegna le più utilizzate nell'ambito delle reti neurali artificiali.

1.5.1 Funzione di attivazione logistica

Una delle scelte più popolari per la funzione di attivazione è la funzione logistica; è una funzione, solitamente utilizzata per i modelli che necessitano di produrre come output una probabilità, che prende in input un qualsiasi valore reale e restituisce in output un valore compreso nell'intervallo [0,1] ($\psi : \mathbb{R} \to [0, 1]$), dove:

$$\psi(z) = L(z) = \frac{1}{1 + e^{-z}} \tag{1.15}$$

la funzione di attivazione logistica passa da uno stato di non attivazione allo stato di attivazione in modo graduale. Tuttavia, come dimostrabile dalla rappresentazione della sua forma funzionale nella Figura 1.7, se i valori di input della funzione sono "troppo" positivi o "troppo" negativi (formalmente all'esterno dell'intervallo [-5,5]), gli output della funzione tenderanno ad ammassarsi sugli estremi dell'intervallo [0,1]; per questo motivo, nelle applicazioni pratiche si utilizzano set di dati standardizzati al fine di renderli *scale invariant*.

1.5.2 Funzione tangente iperbolica (tanh)

L'output della funzione logistica non è simmetrico intorno allo zero; per questo motivo, gli output di tutti i neuroni avranno lo stesso segno, e questo potrebbe rendere l'addestramento della rete neurale più instabile e difficoltoso. Per evitare questo problema, un'alternativa alla funzione logistica è la funzione tangente iperbolica (o tanh); dal punto di vista formale, le due funzioni di attivazione sono molto simili, anche nella forma sigmoidale ma non nel range di variazione che è identificato nell'intervallo [-1,1]:

$$\psi : \mathbb{R} \to [-1, 1] \tag{1.16}$$

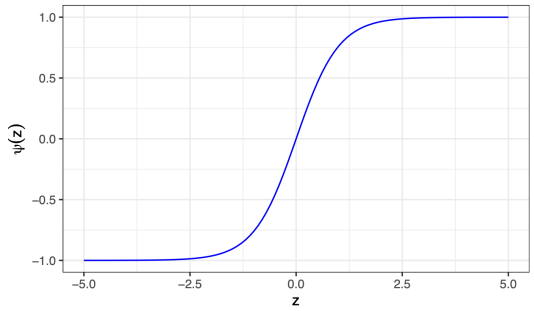


Figura 1.8: Funzione tangente iperbolica (tanh)

dove:

$$\psi(z) = tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \tag{1.17}$$

di fatti la funzione tangente iperbolica è un versione riscalata della funzione logistica nella misura in cui:

$$tanh(z) = 2L(2z) - 1 \tag{1.18}$$

Le principali caratteristiche della funzione di attivazione tanh sono che l'output della funzione è centrato su zero; pertanto, possiamo facilmente mappare i valori di output come fortemente negativi, neutrali o fortemente positivi; inoltre, solitamente viene utilizzata nei neuroni nascosti di una rete neurale poiché la media per il livello nascosto risulta essere 0 o molto vicino a zero e questo aiuta a centrare i dati e rende l'apprendimento per il livello successivo molto più facile. Esattamente come la funzione logistica, anche la funzione tanh satura per piccoli e grandi valori in input, e risulta computazionalmente costosa poiché include un termine esponenziale (e^z).

1.5.3 Funzione ReLU e Leaky ReLU

La funzione di attivazione ReLU (Rectified Linear Unit) è un'ottima alternativa alle funzioni di attivazione logistica e tanh; nasce dalla volontà di ridurre il costo computazionale legato alla presenza degli esponenziali nelle altre funzioni e viene solitamente utilizzata per gli strati nascosti nelle reti di tipo MLP, più di rado viene utilizzata nelle reti RNN, mentre non va utilizzata per i neuroni nello strato di output. Formalmente, la ReLU è definita come:

$$\psi : \mathbb{R} \to [0, \infty) \tag{1.19}$$

dove:

$$\psi(z) = \max\{0, z\} \tag{1.20}$$

in pratica, se il valore in input è pari o maggiore di zero la funzione ReLU restituisce l'input così com'è, mentre se l'input è minore di zero viene restituito zero.

La funzione ReLU è formata da due componenti lineari, ciò la rende molto meno esosa in termini di costo computazionale non presentando termini esponenziali, è una funzione lineare a tratti ma nel complesso è non-lineare.

Uno dei punti deboli dell'utilizzo di questa funzione di attivazione è l'intervallo di variazione che dipende dai valori assunti dall'input che può arrivare fino a $+\infty$. Per delimitare il valore massimo della funzione di attivazione è stata introdotta una variante denominata ReLU6, formalmente definita come:

$$\psi(z) = \min(\max(0, z), 6) \tag{1.21}$$

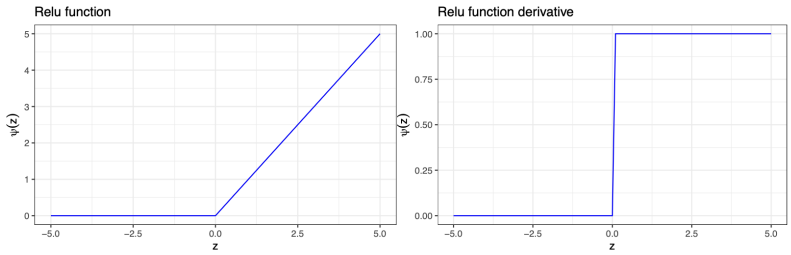


Figura 1.9: Funzione ReLU e sua derivata

il funzionamento è identico a quello della funzione ReLU finchè l'input non risulta pari o superiore a 6, in questi casi verrà restituito come valore della funzione obiettivo 6; caratteristica da cui proviene il nome.

Per la sua conformazione, la funzione ReLU (e la variante ReLU6) non porta con se il problema della scomparsa del gradiente; tuttavia, presenta il problema della *dying ReLU*, che si manifesta nel momento in cui l'input è prevalentemente negativo; quando ciò accade, nello scenario peggiore possibile (con tutti gli input negativi), la rete può "morire", trasformandosi in una funzione costante. Al fine di superare tale limite sono state formalizzate diverse tipologie di funzioni ReLU.

La Leaky ReLU è la prima alternativa nata per risolvere il problema della *dying ReLU*, definita formalmente come:

$$\psi(z) = \begin{cases} \alpha z, & z \leq 0 \\ z, & z > 0 \end{cases}, \quad \alpha = 0.01, 0.02, \ldots \tag{1.22}$$

con derivata pari a:

$$\psi'(z) = \begin{cases} \alpha, & z \leq 0 \\ 1, & z > 0 \end{cases}, \quad \alpha = 0.01, 0.02, \ldots \tag{1.23}$$

È una versione modificata della funzione ReLU predefinita, anche questa funzione non ha il problema del vanishing gradient. Se il valore in input è maggiore di 0, la funzione restituisce l'input stesso; se invece l'input è minore o uguale a 0, restituisce un piccolo valore negativo definito da αz (dove α è una piccola costante, solitamente pari a 0.01, e z è il valore in input). Non presenta componenti lineari con derivata nulla (pendenza zero), il che le consente di evitare il problema della ReLU morente. Il processo di apprendimento è più rapido rispetto a quello della ReLU standard.

1.5.4 Funzione ELU

Una valida alternativa alla funzione ReLU e alle sue varianti è l'Exponential Linear Unit (ELU). L'ELU tende a far convergere la funzione di costo a zero più velocemente e produce risultati più accurati. La funzione è definita come

$$\psi(z) = \begin{cases} \alpha(e^z - 1), & z \leq 0 \\ z, & z > 0 \end{cases}, \quad \alpha = 0.01, 0.02, \ldots \tag{1.24}$$

e ha derivata:

$$\psi(z) = \begin{cases} \alpha e^z, & z \leq 0 \\ 1, & z > 0 \end{cases}, \quad \alpha = 0.01, 0.02, \ldots \tag{1.25}$$

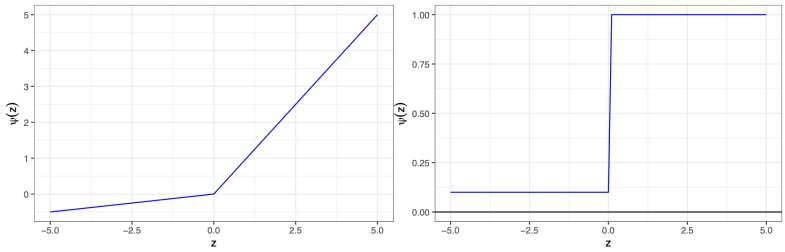


Figura 1.10: Funzione Leaky ReLU e sua derivata

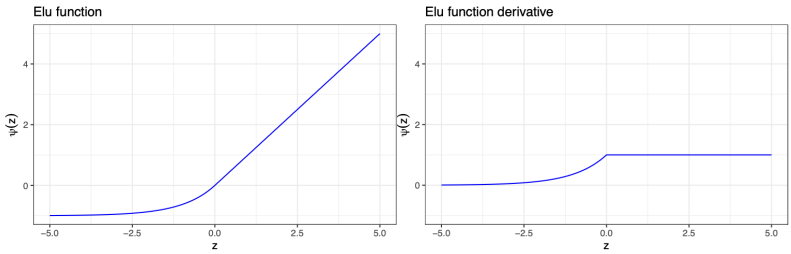


Figura 1.11: Funzione ELU e sua derivata

La costante α deve essere un numero positivo. Quando l'input è negativo, l'ELU diventa gradualmente più "smooth", tendendo a un valore pari a $-\alpha$, mentre le funzioni ReLU agiscono in modo più brusco per input negativi. Uno svantaggio dell'ELU è che, per $z > 0$, può causare un'esplosione dell'attivazione, con un intervallo di output che va da $[0, +\infty]$.

1.5.5 Funzione Swish e Hard-Swish

La funzione Swish è definita moltiplicando la funzione sigmoide per l'input z. Formalmente, è specificata come:

$$\psi(z) = z\frac{1}{1+e^{-z}} \tag{1.26}$$

con derivata pari a:

$$\psi'(z) = \frac{1}{1+e^{-z}} + z\frac{1}{1+e^{-z}}\left(1 - \frac{1}{1+e^{-z}}\right) \tag{1.27}$$

Il grafico della funzione Swish è molto simile al grafico della funzione di attivazione ReLU; la principale differenza risiede nella forma della curva, che è più dolce rispetto alla ReLU. Questa fluidità è importante nella fase di training del modello, permettendo alla funzione di convergere facilmente.

Anche questa funzione di attivazione è utilizzabile quasi esclusivamente per i neuroni dello strato nascosto, il principale lato negativo dell'utilizzo di questa funzione è il costo computazionale elevato a causa del termine esponenziale; tale limite è superabile attraverso l'utilizzo della funzione Hard-Swish.

La funzione Hard-Swish è formalmente definita come segue:

$$\psi(z) = z\frac{ReLU6(z+3)}{6} \tag{1.28}$$

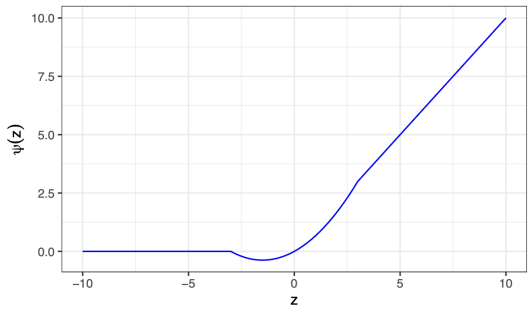


Figura 1.12: Funzione Hard-Swish

il grafico, in questo, caso è identico a quello della funzione Swish ma con il vantaggio del basso costo computazionale grazie alla sostituzione del termine esponenziale con il termine ReLU6 lineare.

1.6 Capacità di approssimazione di funzioni non lineari

Negli ultimi decenni, le reti feedforward multilayer si sono dimostrate piuttosto efficaci in numerose applicazioni diverse; diversi articoli e pubblicazioni riportano infatti come le loro prestazioni siano almeno pari a quelle dei modelli tradizionali in diverse applicazioni statistiche.

Questo successo ha portato, nel corso degli anni, diversi ricercatori a intraprendere un'analisi rigorosa delle proprietà matematiche che consentono alle reti neurali feedforward di ottenere buoni risultati nelle applicazioni in numerosi campi diversi. Tali risultati sono noti come teoremi di approssimazione universali (*Universal Approximation Theorems*, UATs) e coprono un periodo che va dal 1989, con l'articolo fondamentale di Cybenko (1989) [1], fino al 2020 con l'articolo di A. Heinecke, J. Ho e W. Hwang (2020) [2].

I teoremi di approssimazione universale affermano che le reti neurali feed-forward con un singolo strato nascosto, contenente un numero finito di neuroni, sono in grado di approssimare qualsiasi funzione regolare di interesse g, a qualsiasi livello di accuratezza desiderato, in condizioni generali relative alle funzioni di attivazione. Tali teoremi si basano fondamentalmente su un trade-off tra condizioni sulla funzione da approssimare g e la funzione di attivazione ψ; più deboli saranno le assunzioni su g e più forti saranno le assunzioni su ψ, e viceversa. Vediamo di seguito il teorema stipulato nel lavoro di Cybenko:

> **Teorema 1.1** *Sia ψ una funzione di attivazione sigmoidale. Per qualsiasi funzione continua $g : \mathbb{R}^d \to \mathbb{R}$ e per ogni $\epsilon > 0$, esiste una dimensione finita del layer nascosto r_0 tale che per ogni $r > r_0$:*
>
> $$\sup_{\mathbf{x} \in K} |f_r(\mathbf{x}) - g(\mathbf{x})| < \epsilon$$
>
> *dove K è un sottoinsieme compatto di $\mathbb{R}^d$.*

I teoremi di approssimazione universale sono importanti risultati teorici che indicano che le reti neurali possono rappresentare molte funzioni interessanti se dotate di pesi adeguati; tuttavia, in genere non forniscono una procedura per la determinazione dei pesi, ma si limitano ad affermare che tale procedura è possibile. La fase di selezione del modello (scelta della topologia e architettura di rete) rimane una fase cruciale e complessa; una volta selezionata la topologia della rete, la fase di addestramento rimane comunque un compito impegnativo e computazionalmente costoso.

1.7 L'addestramento della rete come problema di ottimizzazione

Una volta fissato r, è possibile introdurre delle funzioni di perdita (non necessariamente quadratiche) per tenere conto della natura specifica del problema in esame:

$$\mathcal{R}(\theta) = \frac{1}{n}\sum_{i=1}^{n}\mathcal{L}(Y_i, f_r(x_i, \theta)) \tag{1.29}$$

dove $\mathcal{L}$ è una funzione di perdita appropriata e ben definita, che misura la perdita individuale per un dato valore del vettore di parametri θ. Data la funzione di perdita, il problema si riduce alla risoluzione del seguente problema di ottimizzazione:

$$\hat{\theta}_n = \arg\min_{\theta} \frac{1}{n}\sum_{i=1}^{n}\mathcal{L}(Y_i, f_r(x_i, \theta)) \tag{1.30}$$

Tali problemi di ottimizzazione sono difficili da risolvere numericamente nell'ambito delle reti neurali, in quanto incorrono in due diverse tipologie di cause:

1. La dimensione dello spazio parametrico elevata, che cresce velocemente con il numero di input d, e con il numero di neuroni negli strati nascosti r. In particolare, la grandezza dello spazio parametrico, indicata con Θ, è pari, per una rete neurale feedforward, a $r(d+2)+1$;

2. Le caratteristiche non ottimali della funzione obiettivo che, a causa della presenza di minimi locali, può bloccare il processo di stima, impedendo così di raggiungere il punto di ottimo globale.

1.7.1 Il ruolo degli algoritmi di ottimizzazione

Nel corso degli anni sono stati sviluppati diversi algoritmi di ottimizzazione che permettessero di affrontare e superare le problematiche proposte, di seguito se ne analizza una parte largamente utilizzata in letteratura e nelle applicazioni pratiche.

Il primo algoritmo di ottimizzazione trattato è il metodo di Newton; per poterlo applicare è necessario supporre che la funzione $\mathcal{R}$ sia differenziabile due volte e che quindi hessiano e gradiente siano calcolabili. Fissato θ_0 il vettore iniziale dei pesi, utilizziamo l'espansione in serie di Taylor del secondo ordine per approssimare la funzione $\mathcal{R}$ localmente:

$$\mathcal{R}(\theta) = \mathcal{R}(\theta_t) + \nabla\mathcal{R}(\theta_t)^T(\theta-\theta_t) + \frac{1}{2}(\theta-\theta_t)^T H(\mathcal{R}(\theta_t))(\theta-\theta_t) + Remainder \tag{1.31}$$

dove $\nabla\mathcal{R}(\theta_t)$ è il gradiente e dove $H(\mathcal{R}(\theta_t))$ è la matrice hessiana, entrambi valutati nel punto θ_t. Il vettore dei parametri $\hat{\theta}_t$ che minimizza $\mathcal{R}(\theta)$ annulla la derivata del primo ordine dell'espansione precedente. Non prendendo in considerazione il termine di errore e ponendo la derivata di $\mathcal{R}(\theta)$ rispetto a θ uguale a zero:

$$\nabla\mathcal{R}(\theta_t) + H(\mathcal{R}(\theta_t))(\hat{\theta}-\theta_t) = 0 \tag{1.32}$$

si ottiene:

$$\hat{\theta} = \theta_t - [H(\mathcal{R}(\theta_t))]^{-1}\nabla\mathcal{R}(\theta_t) \tag{1.33}$$

Questa relazione suggerisce un algoritmo di ottimizzazione che aggiorna iterativamente l'insieme dei pesi dei neuroni come riportato nell'Algoritmo 1.

Il numero di iterazioni $\mathcal{T}$ è scelto in modo che la variazione della perdita totale $\mathcal{R}$ dopo $\mathcal{T}$ passi, scenda al di sotto di un criterio di tolleranza debitamente scelto. Il metodo di Newton è

Algorithm 1 - Metodo di Newton

Inizializzazione
  Attribuisci casualmente i pesi a ciascun neurone: θ_0
Procedura principale
for $t = 0$ a epoca massima, $\mathcal{T}$ **do**
  1. Calcola il gradiente $\nabla\mathcal{R}(\theta_t)$
  2. Calcola la matrice Hessiana $H(\mathcal{R}(\theta_t))$
  3. Inverti l'Hessiana e aggiorna il vettore dei pesi:
    $\theta_{t+1} = \theta_t - H(\mathcal{R}(\theta_t))^{-1}\nabla\mathcal{R}(\theta_t)$
end for

indicato per minimizzare principalmente funzioni obiettivo convesse, risulta particolarmente costoso dal punto di vista computazionale poiché il principale problema è l'inversione della matrice Hessiana, oltre a dipendere dal punto di inizializzazione dei pesi θ_0. Nelle condizioni di cui sopra, risulta essere un metodo molto efficiente riuscendo a convergere ad una soluzione approssimata in poche iterazioni.

Una soluzione alternativa al metodo di Newton è offerta da una classe di algoritmi di ottimizzazione noti come *Gradient Descent* (discesa del gradiente). In questo caso si utilizza solo il gradiente, interrompendo l'espansione di Taylor al primo ordine:

$$\mathcal{R}(\theta) = \mathcal{R}(\theta_t) + \nabla\mathcal{R}(\theta_t)^T(\theta - \theta_t) + Remainder \tag{1.34}$$

L'aggiornamento dei pesi θ_t avviene attraverso la variazione di un piccolo passo nella direzione opposta a quella del gradiente. L'iterazione chiave è data da:

$$\theta_{t+1} = \theta_t - \eta\nabla\mathcal{R}(\theta_t) \tag{1.35}$$

la procedura è descritta dall'algoritmo 2:

Algorithm 2 - Gradient Descent

Inizializzazione
  Attribuisci casualmente i pesi a ciascun neurone: θ_0
  Seleziona il passo iniziale: η_0
Procedura principale
for $t = 0$ a epoca massima, $\mathcal{T}$ **do**
  1. Calcola il gradiente $\nabla\mathcal{R}(\theta_t)$
  2. Aggiorna la dimensione del passo: $\eta_t = h(\eta_0, t)$
  3. Aggiorna il vettore dei pesi: $\theta_{t+1} = \theta_t - \eta_t\nabla\mathcal{R}(\theta_t)$
  4. Interrompi se $\|\nabla\mathcal{R}(\theta_{t+1})\| < \epsilon$
end for

Per l'aggiornamento dei pesi risulta fondamentale il valore del tasso di apprendimento η, anche detto *learning rate*; in particolare, la grandezza dei passi che la discesa del gradiente compie verso il minimo locale dipende dal tasso di apprendimento, che determina la velocità con cui ci si avvicina ai pesi ottimali. Per fare in modo che la discesa del gradiente raggiunga il minimo locale, è necessario impostare il tasso di apprendimento a un valore appropriato, che non sia né troppo basso né troppo alto; se i passi sono troppo grandi, il metodo potrebbe non riuscire a convergere al minimo locale, poiché rischia di saltare oltre il punto ottimale, oscillando avanti e indietro lungo la funzione, se, invece, il tasso di apprendimento è troppo

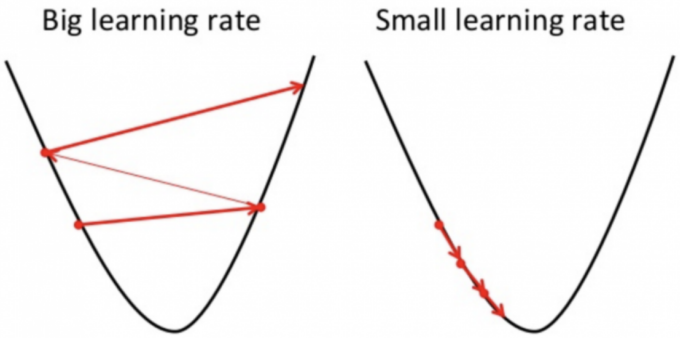


Figura 1.13: Tasso di apprendimento grande vs piccolo

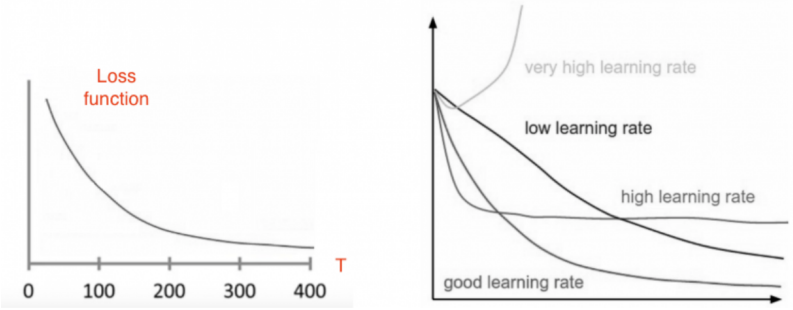


Figura 1.14: Comparazione del processo di apprendimento

piccolo, la discesa del gradiente raggiungerà comunque il minimo locale, ma impiegherà molto più tempo (vedi Figura 1.13).

Un buon modo per assicurarsi che la discesa del gradiente funzioni correttamente è tracciare la funzione di costo mentre l'ottimizzazione è in corso; ciò permette di osservare il valore della funzione di costo a ogni iterazione della discesa del gradiente e aiuta a capire quanto sia appropriato il tasso di apprendimento scelto. In particolare, è possibile provare diversi valori del tasso di apprendimento e riportare i risultati in un unico grafico, come mostrato nella Figura 1.14.

Il secondo algoritmo della classe Gradient Descent che prendiamo in considerazione è la sua versione stocastica; questa nasce dalla necessità di ridurre il costo computazionale del calcolo del gradiente introducendo un elemento di casualità nel dataset; infatti, ad ogni iterazione, lo *Stochastic Gradient Descent* (SGD) mescola casualmente i dati e aggiorna i parametri su un singolo campione ridotto scelto a caso, invece di effettuare un aggiornamento sull'intero dataset. Al posto di calcolare il gradiente completo, si utilizza un gradiente stimato su un sottoinsieme di B osservazioni scelte tra le n che compongono il dataset originale. Il numero di osservazioni B deve essere scelto in modo equilibrato, non troppo piccolo per evitare instabilità numerica, ma nemmeno troppo grande per non rendere il calcolo troppo oneroso; le dimensioni comuni dei mini-batch variano tra 48 e 256, ma non esiste una regola precisa, poiché dipende dalle diverse applicazioni. La procedura è descritta dall'algoritmo 3 espresso in pseudo-codice:

L'algoritmo sfrutta la proprietà di non distorsione dello stimatore per il vero gradiente

Algorithm 3 - Stochastic Gradient Descent

Inizializzazione
  Attribuisci casualmente i pesi a ciascun neurone: θ_0
  Seleziona il passo iniziale: η_0
Procedura principale
for $t = 0$ a epoca massima, $\mathcal{T}$ **do**
  1. Mescola B data points dal dataset originale. Sia $\mathcal{B}$ questo dataset.
  2. Approssima il gradiente $\nabla\tilde{\mathcal{R}}(\theta_t) = \frac{1}{B}\sum_{i\in\mathcal{B}} \nabla\mathcal{L}_i(\theta_t)$
  3. Aggiorna la dimensione del passo: $\eta_t = h(\eta_0, t)$
  4. Aggiorna il vettore dei pesi: $\theta_{t+1} = \theta_t - \eta_t\nabla\tilde{\mathcal{R}}(\theta_t)$
  5. Interrompi se $\|\nabla\tilde{\mathcal{R}}(\theta_{t+1})\| < \epsilon$
end for

$\nabla\tilde{\mathcal{R}}(\theta_t)$. Valutandone il valore atteso, infatti, è possibile scrivere che:

$$E[\nabla\tilde{\mathcal{R}}(\theta_t)] = E\left[\frac{1}{B}\sum_{i\in\mathcal{B}} \nabla\mathcal{L}_i(\theta_t)\right] = \nabla\mathcal{R}(\theta_t) \tag{1.36}$$

inoltre, la varianza di questo stimatore è inversamente proporzionale rispetto a B, per cui all'aumentare della grandezza del batch diminuirà la varianza dello stimatore.

Un'ulteriore variante del SGD è lo Stochastic Gradient Descent con Momentum; questa tecnica introduce un termine per migliorare la stabilità e la velocità di convergenza, riducendo le oscillazioni e facilitando il percorso verso il minimo della funzione di perdita. La procedura è descritta dall'algoritmo 4 espresso in pseudo-codice:

Algorithm 4 - Stochastic Gradient Descent con Momentum

Inizializzazione
  Attribuisci casualmente i pesi a ciascun neurone: θ_0
  Seleziona il passo iniziale: η_0
Procedura principale
for $t = 0$ a epoca massima, $\mathcal{T}$ **do**
  1. Calcola il gradiente $\nabla\mathcal{R}(\theta_t)$
  2. Aggiorna la dimensione del passo: $\eta_t = h(\eta_0, t)$
  3. Aggiorna il vettore dei pesi: $\theta_{t+1} = \theta_t - \eta_t\nabla\mathcal{R}(\theta_t) + \gamma(\theta_t - \theta_{t-1})$
  4. Interrompi se $\|\nabla\mathcal{R}(\theta_{t+1})\| < \epsilon$
end for

Un valore tipico per γ è 0.9, mentre per $\gamma = 0$ si ottiene lo SGD senza momentum. Quando il gradiente punta sempre nella stessa direzione per diversi passi consecutivi, il termine momentum lo rafforza, ciò aiuta ad accelerare l'apprendimento nelle direzioni più rilevanti migliorando la velocità di convergenza. Quando il gradiente cambia direzione frequentemente, significa che la superficie della funzione di perdita è molto irregolare, in questi casi lo SGD classico potrebbe oscillare troppo tra valori vicini, rendendo difficile la discesa verso il minimo; il momentum smorza questi cambiamenti evitando che l'algoritmo segua ogni piccola variazione e rendendo il percorso più stabile.

Come già detto in precedenza, il metodo di Newton è indicato solo per minimizzare funzioni obiettivo convesse; inoltre, risulta particolarmente costoso dal punto di vista computazionale, poiché il principale problema è l'inversione della matrice Hessiana. Per far

fronte a questa problematica, sono stati sviluppati i metodi quasi-Newton in cui, invece di calcolare l'Hessiana effettiva, essa viene approssimata con una matrice definita positiva B, che viene aggiornata di iterazione in iterazione utilizzando le informazioni calcolate dai passi precedenti. In letteratura sono stati sviluppati vari metodi quasi-Newton che si differenziano per il modo in cui la matrice Hessiana approssimata, B, viene aggiornata ad ogni iterazione; ad oggi, il metodo quasi-Newton più ampiamente utilizzato è il metodo BFGS (Broyden–Fletcher–Goldfarb–Shanno).

La condizione quasi-Newton imposta sulla matrice B_{t+1}, in questo metodo, è:

$$B_{t+1}(\theta_{t+1} - \theta_t) = \nabla\mathcal{R}(\theta_{t+1}) - \nabla\mathcal{R}(\theta_t) \tag{1.37}$$

dove $z_t = \nabla\mathcal{R}(\theta_{t+1}) - \nabla\mathcal{R}(\theta_t)$ e $s_t = \theta_{t+1} - \theta_t$, implicando che $B_{t+1} \cdot s_t = z_t$. La procedura è descritta dall'algoritmo 5 espresso in pseudo-codice:

Algorithm 5 - BFGS (Broyden–Fletcher–Goldfarb–Shanno)

Inizializzazione
  Attribuisci casualmente i pesi a ciascun neurone: θ_0
  Imposta una matrice Hessiana appropriata: B_0
Procedura principale
for $t = 0$ a epoca massima, $\mathcal{T}$ **do**
  1. Ottieni una direzione p_t risolvendo $B_t p_t = -\nabla\mathcal{R}(\theta_t)$
  2. Esegui un'ottimizzazione unidimensionale (ricerca lineare) per trovare una dimensione del passo accettabile α_t nella direzione trovata nel primo passo. Se viene eseguita una ricerca lineare esatta, allora $\alpha_t = \arg\min f(x_t + \alpha p_t)$.
  3. Imposta $s_t = \alpha_t p_t$ e aggiorna $\theta_{t+1} = \theta_t + s_t$
  4. $y_t = \nabla f(x_{t+1}) - \nabla f(x_t)$
  5. $B_{t+1} = B_t + \frac{z_t z_t^T}{z_t^T s_t} - \frac{B_t s_t s_t^T B_t^T}{s_t^T B_t s_t}$
end for

1.7.2 La tecnica della backpropagation per la computazione del gradiente

Il passaggio fondamentale del metodo del Gradient Descent risiede nel calcolo del gradiente, ed è in tale contesto che l'algoritmo di backpropagation (retropropagazione) risulta di particolare utilità. Al fine di comprendere appieno tale meccanismo, è opportuno richiamare la nota regola della catena per il calcolo delle derivate. La regola della catena (*chain rule*) è una formula che esprime la derivata della composizione di due funzioni differenziabili $f(\cdot)$ e $g(\cdot)$ in termini delle derivate stesse di $f(\cdot)$ e $g(\cdot)$. Più precisamente, se $h = f \circ g$ è la funzione tale per cui

$$h(x) = f(g(x)) \tag{1.38}$$

per ogni x, allora la regola della catena assume, secondo la notazione di Lagrange, la seguente forma:

$$h'(x) = f'(g(x))g'(x). \tag{1.39}$$

Tale regola può inoltre essere espressa avvalendosi della notazione di Leibniz. Qualora una variabile z dipenda dalla variabile y, la quale a sua volta dipende dalla variabile x (ovvero, y e z costituiscono variabili dipendenti), ne consegue che anche z dipenderà da x attraverso la variabile intermedia y, ossia:

$$x \to y \to z \tag{1.40}$$

in questo caso, la regola della catena si esprime come:

$$\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}, \tag{1.41}$$

e

$$\left.\frac{dz}{dx}\right|_x = \left.\frac{dz}{dy}\right|_{y(x)} \left.\frac{dy}{dx}\right|_x, \tag{1.42}$$

al fine di indicare in quali punti le derivate debbano essere valutate.

Questa regola risulta di fondamentale importanza per il calcolo del gradiente all'interno delle reti neurali; ad esempio, se si considera una rete neurale estremamente semplice, dotata di un singolo neurone di input, un singolo neurone di output e una funzione di attivazione logistica, la cui espressione funzionale è la seguente:

$$y = c_1\psi(w_1x + w_0) + c_0 \tag{1.43}$$

tale funzione definisce la fase forward (o fase di propagazione in avanti):

$$x \to \boxed{w_1x + w_0} \to \boxed{\psi(w_1x + w_0)} \to \boxed{c_1\psi(w_1x + w_0) + c_0} \to y \tag{1.44}$$

La fase forward costituisce il risultato della composizione di diverse funzioni:

1. $l = w_1x + w_0$
2. $h = \psi(l)$
3. $y = c_1h + c_0$

Fissato un valore per x, sia esso x_i, e data una specifica configurazione per i valori dei parametri, si ottiene il valore predetto dalla rete:

$$x_i \to \boxed{w_1x_i + w_0} \to \boxed{\psi(w_1x_i + w_0)} \to \boxed{c_1\psi(w_1x_i + w_0) + c_0} \to \hat{y}_i \tag{1.45}$$

adottando una funzione di costo quadratica (*quadratic loss*), è possibile confrontare il valore osservato y_i con quest'ultimo, definendo così la funzione di costo i-esima:

$$\mathcal{L}_i(w_0, w_1, c_0, c_1) = \frac{1}{2}\left(y_i - \hat{y}_i(w_0, w_1, c_0, c_1)\right)^2 \tag{1.46}$$

La funzione di costo totale, invece, è pertanto espressa come:

$$\mathcal{R}(w_0, w_1, c_0, c_1) = \frac{1}{n}\sum_{i=1}^{n} \mathcal{L}_i(w_0, w_1, c_0, c_1) \tag{1.47}$$

Il gradiente di tale funzione (ovvero il vettore delle derivate rispetto ai parametri) quantifica l'impatto di una variazione dei parametri medesimi sulla funzione di costo. Sfruttando una nota proprietà degli operatori di derivazione (la derivata di una somma equivale alla somma delle singole derivate), l'analisi si focalizza sulle derivate del singolo addendo della sommatoria, $\mathcal{L}_i$, calcolate rispetto a tutti i parametri coinvolti.

Si procede applicando la regola della catena per il calcolo delle derivate:

$$\begin{aligned}\frac{\partial \mathcal{L}_i}{\partial w_1} &= \frac{\partial}{\partial w_1}\left\{\frac{1}{2}\left(y_i - \hat{y}_i(w_0, w_1, c_0, c_1)\right)^2\right\} \\ &= (y_i - \hat{y}_i)\frac{\partial \hat{y}_i}{\partial w_1} \\ &= (y_i - \hat{y}_i)\frac{\partial \hat{y}_i}{\partial \psi}\frac{\partial \psi}{\partial w_1} \\ &= (y_i - \hat{y}_i)\frac{\partial \hat{y}_i}{\partial \psi}\frac{\partial \psi}{\partial l}\frac{\partial l}{\partial w_1}\end{aligned} \tag{1.48}$$

Si entra, a questo punto, nella fase backward, dove vengono calcolate le singole derivate parziali per ciascuno dei parametri della rete in modo da valutarne il loro impatto sulla funzione di perdita complessiva:

$$\frac{\partial l}{\partial w_1} = \frac{\partial}{\partial w_1}\{w_1 x_i + w_0\} = x_i \tag{1.49}$$

$$\frac{\partial \psi}{\partial l} = \frac{\partial}{\partial l}\left\{\frac{1}{1+e^{-l}}\right\} = \psi(l)[1 - \psi(l)] \tag{1.50}$$

tale risultato costituisce una ben nota proprietà della funzione logistica. Infine, si ottiene:

$$\frac{\partial \hat{y}_i}{\partial \psi} = \frac{\partial}{\partial \psi}\{c_1 \psi + c_0\} = c_1 \tag{1.51}$$

Componendo le singole derivate, la componente del gradiente relativa al peso w_1 risulta essere:

$$\frac{\partial \mathcal{L}_i(\theta)}{\partial w_1} = (y_i - \hat{y}_i(w_1, w_0, c_1, c_0))\psi(w_1 x_i + w_0)(1 - \psi(w_1 x_i + w_0))c_1 x_i \tag{1.52}$$

Le rimanenti componenti possono essere ricavate mediante un procedimento analogo. In particolare, la componente del gradiente associata al parametro w_0 è definita come:

$$\frac{\partial \mathcal{L}(\theta)}{\partial w_0} = [y_i - \hat{y}_i(w_1, w_0, c_1, c_0)]\psi(w_1 x_i + w_0)[1 - \psi(w_1 x_i + w_0)]c_1 \tag{1.53}$$

Per quanto concerne c_1, uno dei pesi afferenti allo strato di output (*output layer*), la corrispondente componente del gradiente è:

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial c_1} &= \frac{\partial}{\partial c_1}\left\{\frac{1}{2}(y_i - \hat{y}_i(w_0, w_1, c_0, c_1))^2\right\} \\ &= (y_i - \hat{y}_i)\frac{\partial \hat{y}_i}{\partial c_1} \\ &= (y_i - \hat{y}_i)\psi(w_1 x_i + w_0)\end{aligned} \tag{1.54}$$

Di conseguenza:

$$\frac{\partial \mathcal{L}(\theta)}{\partial c_1} = [y_i - \hat{y}_i(w_0, w_1, c_0, c_1)]\psi(w_1 x_i + w_0) \tag{1.55}$$

Infine, derivando rispetto a c_0, si ottiene semplicemente:

$$\begin{aligned}\frac{\partial \mathcal{L}_i}{\partial c_0} &= \frac{\partial}{\partial c_0}\left\{\frac{1}{2}(y_i - \hat{y}_i(w_0, w_1, c_0, c_1))^2\right\} \\ &= (y_i - \hat{y}_i)\frac{\partial \hat{y}_i}{\partial c_0} \\ &= (y_i - \hat{y}_i) \cdot 1\end{aligned} \tag{1.56}$$

$$\frac{\partial \mathcal{L}_i(\theta)}{\partial c_0} = [y_i - \hat{y}_i(w_0, w_1, c_0, c_1)] \tag{1.57}$$

Il processo di apprendimento (o di stima) si articola, quindi, in un ciclo iterativo costituito da fasi di forward e fasi di backward:

1. Nella fase forward (di propagazione in avanti), l'output della rete viene calcolato sulla base di una specifica configurazione (provvisoria) dei valori dei pesi.

2. Nella fase backward (di retropropagazione), a seguito della valutazione della funzione di costo, l'errore viene retropropagato (avvalendosi del calcolo del gradiente) al fine di determinare un nuovo aggiornamento per i valori dei parametri, con l'obiettivo di minimizzare la perdita complessiva.

Il termine *backpropagation* denota esclusivamente la procedura di calcolo del gradiente e non la sua successiva applicazione nel processo di ottimizzazione; ciononostante, nella prassi comune, tale dicitura viene utilizzata per includere anche il metodo del *Gradient Descent* (GD), impiegato per derivare i valori ottimali dei pesi.

2. Le reti neurali nel contesto delle serie storiche non lineari

2.1 Modelli per serie storiche non lineari

L'obiettivo dell'analisi delle serie storiche consiste nel comprendere le dinamiche sottostanti a un insieme di dati osservati e temporalmente ordinati. In diversi contesti, i modelli lineari (AR, MA, ARMA e ARIMA) forniscono prestazioni insoddisfacenti, rendendo necessaria la specificazione di modelli non lineari per descrivere più accuratamente la dinamica delle serie storiche prese in esame.

Le possibili tipologie di elaborazione delle serie storiche non lineari includono:

1. la modellazione del processo di generazione dei dati (DGP, *Data Generating Process*) di una serie storica osservata (analisi delle serie storiche);
2. la previsione dei valori futuri di Y_t (*forecasting*);
3. la classificazione di una serie all'interno di un insieme discreto di classi (*time series clustering*).

In molteplici applicazioni reali relative alle serie storiche, il meccanismo generatore dei dati risulta essere fortemente complesso e/o completamente ignoto; tali dati non possono essere descritti in modo adeguato mediante semplici equazioni analitiche. Inoltre, qualora la serie storica osservata sia influenzata da trend e stagionalità, la stima e/o la rimozione di tali componenti può risultare complessa dal punto di vista applicativo.

All'atto pratico, l'impiego degli strumenti statistici tradizionali per l'analisi delle serie storiche richiede una notevole esperienza e competenza per selezionare il modello più idoneo a uno specifico insieme di dati; pertanto, il principale vantaggio offerto dalle reti neurali risiede, ancora una volta, nella mancata necessità di specificare a priori l'esatta natura della relazione (lineare, non lineare, stagionalità, trend) intercorrente tra input e output. Gli strati nascosti (*hidden layers*) di una rete neurale eliminano l'esigenza di definire preventivamente la struttura del meccanismo di generazione dei dati.

Prima delle reti neurali sono stati identificati diversi modelli parametrici per serie storiche atti a gestire la non linearità dei dati, tra questi il modello più generale è senza dubbio il modello autoregressivo non lineare; per definirlo in modo formale è necessario definire il processo stocastico $\{Y_t, t \in \mathbb{Z}\}$ come segue:

$$Y_t = g(Y_{t-1}, Y_{t-2}, \ldots, Y_{t-p}) + \epsilon_t \tag{2.1}$$

dove g è una funzione incognita appartenente a una determinata classe di regolarità (*smoothness*).

Tale modello è valido sotto la seguente ipotesi di causalità:

$$\epsilon_t \overset{\text{iid}}{\sim} (0, \sigma^2) \tag{2.2}$$

tale per cui ϵ_t risulti indipendente da $\{Y_s, s < t\}$, per ogni t. L'ipotesi garantisce che:

$$E[Y_t | Y_s, s < t] = g(Y_{t-1}, Y_{t-2}, \ldots, Y_{t-p}) \tag{2.3}$$

ossia, che il valore atteso rappresenta il predittore ottimale di Y_t condizionatamente a $\{Y_s, s < t\}$. In questo contesto, l'ottimalità è intesa rispetto all'errore quadratico medio (MSE, *Mean*

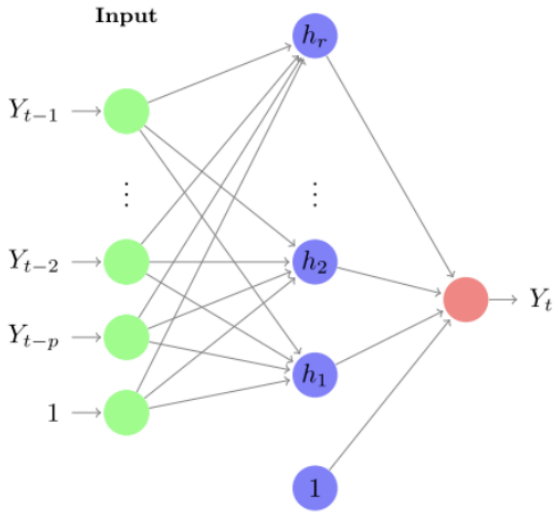


Figura 2.1: Rete neurale autoregressiva di ordine p, NNAR(p,r)

Squared Error) di previsione. Il modello descritto configura un processo autoregressivo puramente non lineare di ordine p (solitamente indicato come $NAR(p)$).

Oltre al modello $NAR(p)$, integrabile con delle variabili esplicative ($NARX(p)$), esistono molte altre configurazioni parametriche per serie storiche non lineari; ne sono un esempio i modelli Exponential Autoregressive (EXPAR), Threshold Autoregressive (TAR), Self-exiting Threshold Autoregressive (SETAR) e Smooth Transition Autoregressive (STAR).

2.2 Modelli di reti neurali per serie storiche non lineari

Al fine di stimare la funzione incognita g, è possibile impiegare una rete neurale dotata di p neuroni di input (uno per ciascun ritardo, o *lag*) e di un singolo neurone di output. Sia $Y_1, Y_2, \ldots, Y_T$ una serie storica osservata e sia:

$$\begin{aligned} Y_t &= f_r(\mathbf{y}_{t-1}, \theta) + \epsilon \\ &= c_0 + \sum_{k=1}^{r} c_k \psi \left(\sum_{j=1}^{p} w_{kj} Y_{t-j} + w_{k0} \right) + \epsilon_t \end{aligned} \tag{2.4}$$

dove $\mathbf{y}_{t-1} = Y_{t-1}, Y_{t-2}, \ldots, Y_{t-p}$.

Notando che un modello autoregressivo basato su reti neurali rappresenta una generalizzazione di un modello AR: una rete neurale caratterizzata da una connessione diretta tra input e output (ovvero priva di strati nascosti) si riduce a un modello $AR(p)$. Un modello a rete neurale con p ritardi viene denotato come $NNAR(p, r)$ (p neuroni di input e r neuroni nascosti).

In presenza di dati stagionali, risulta utile includere tra le variabili di input anche gli ultimi valori osservati relativi alla medesima stagione. Sia m il periodo della componente stagionale, con $m = 12$ nel caso di dati mensili, $m = 4$ per dati trimestrali, e così via. Un modello $NNAR(p, P, r)_m$ denota una rete neurale caratterizzata da r neuroni nascosti, un output pari a Y_t e i seguenti input:

$$(Y_{t-1}, Y_{t-2}, \ldots, Y_{t-p}, Y_{t-m}, Y_{t-2m}, \ldots, Y_{t-Pm})$$

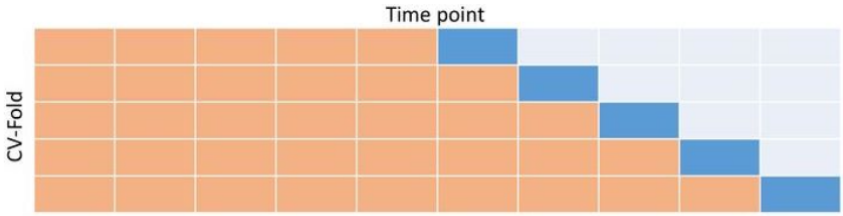


Figura 2.2: Time Series Cross-Validation

Esattamente come la controparte non stagionale, anche un modello $NNAR(p, P, 0)_m$ è equivalente a un modello $ARIMA(p, 0, 0)(P, 0, 0)_m$, ad eccezione dell'assenza delle restrizioni sui parametri che assicurano la stazionarietà.

2.2.1 Previsioni con reti neurali non lineari autoregressive

Affrontiamo ora il problema delle previsioni con i modelli $NNAR$, in particolare la previsione a un passo avanti (One-step ahead).

Sia il DGP:

$$Y_t = g(\mathbf{y}_{t-1}) + \epsilon_t \tag{2.5}$$

dove ϵ_t è una sequenza di variabili casuali iid con media zero e varianza finita σ^2 e $\mathbf{y}_{t-1} = (Y_{t-1}, Y_{t-2}, \ldots, Y_{t-p})$. Sia Y_{T+1} il valore della serie previsto un passo in avanti; sotto l'ipotesi di causalità, il predittore MSE ottimale per Y_{T+1} dato $\{Y_s, s < T\}$ è dato da:

$$E[Y_{T+1}|Y_T, Y_{T-1}, \ldots, Y_{T-p+1}] = g(Y_T, Y_{T-1}, \ldots, Y_{T-p+1}) \tag{2.6}$$

La funzione g è sconosciuta, ma può essere stimata utilizzando una rete neurale. Una stima della previsione per Y_{T+1} può essere costruita come:

$$\begin{aligned} \hat{Y}_{T+1|T} &= \hat{g}(Y_T, Y_{T-1}, \ldots, Y_{T-p+1}) \\ &= f_r(Y_T, Y_{T-1}, \ldots, Y_{T-p+1}; \hat{\theta}) \end{aligned} \tag{2.7}$$

2.3 Cross-validation per serie storiche

Per i dati relativi a serie temporali, la cross-validation standard non può essere applicata in modo acritico. In tale contesto, non risulta possibile selezionare campioni in modo casuale per assegnarli al set di addestramento (training set) o al set di test (test set). Nelle serie storiche è intrinsecamente presente una dipendenza temporale tra le osservazioni che non può essere ignorata, bensì deve essere rigorosamente preservata durante le fasi di addestramento e di valutazione.

Quando si elaborano dati di natura temporale, l'approccio metodologico ottimale per la validazione del modello è la *rolling window cross-validation*; in questa procedura si definisce una sequenza di test set, ciascuno costituito da una singola osservazione, come mostrato nella Figura 2.2.

Il set di addestramento corrispondente è costituito esclusivamente dalle osservazioni registrate antecedentemente a quella che forma il test set; di conseguenza, nessuna osservazione futura può essere impiegata per la formulazione della previsione (*forecast*). Poiché non è possibile ottenere una previsione affidabile basandosi su un set di addestramento di dimensioni eccessivamente ridotte, le primissime osservazioni della serie non vengono utilizzate come set di test. Nell'approccio precedentemente descritto, le previsioni si basano su una finestra

in espansione (*expanding window*); in alternativa, è possibile prendere in considerazione l'utilizzo di finestre di dimensione fissa.

2.3.1 Misure di accuratezza previsiva

L'accuratezza di un modello di previsione può essere determinata esclusivamente valutando le sue prestazioni su dati nuovi, non utilizzati durante la fase di addestramento, poiché la dimensione dei residui calcolati sul training set non costituisce un indicatore affidabile dell'entità dei reali errori di previsione. Nell'ambito di procedure di convalida come la time series cross-validation, le metriche di accuratezza vengono calcolate basandosi sull'errore di previsione, definito come la differenza tra il valore osservato nel set di test e la relativa previsione generata dal modello ($e_{T+h} = Y_{T+h} - \hat{Y}_{T+h|T}$), dove h rappresenta l'orizzonte previsivo.

Il Mean Absolute Error (MAE) è una misura dipendente dalla scala che si calcola come la media dei valori assoluti degli errori di previsione; formalmente definita come segue:

$$MAE = \frac{1}{H}\sum_{h=1}^{H}|e_h| \tag{2.8}$$

dove H è la numerosità delle osservazioni nel set di test.

Questa metrica risulta particolarmente popolare per la sua semplicità di comprensione e computazione, rendendola la scelta preferibile quando si devono confrontare diversi metodi di previsione applicati a una singola serie storica o a più serie che condividono le stesse unità di misura. Sotto il profilo statistico, un modello che mira a minimizzare il MAE produrrà previsioni che sono ottimali nel senso della mediana.

Il Root Mean Squared Error (RMSE) è anch'esso una misura dipendente dalla scala, calcolata come la radice quadrata della media dei quadrati degli errori di previsione. Viene formalmente definita come:

$$RMSE = \sqrt{\frac{1}{H}\sum_{h=1}^{H}e_h^2} \tag{2.9}$$

A differenza del MAE, l'RMSE tende a penalizzare in modo più severo gli errori di maggiore entità a causa dell'elevamento al quadrato, rendendolo una metrica meno intuitiva da interpretare ma estremamente diffusa. L'utilizzo dell'RMSE è preferibile nei contesti in cui l'obiettivo è ottenere previsioni ottimali nel senso della media. Entrambi questi indici sono sempre maggiori o uguali a zero, dove il valore zero indica una previsione perfetta e valori crescenti denotano una minore accuratezza.

Per ovviare ai limiti delle misure dipendenti dalla scala, specialmente quando è necessario confrontare le performance previsive tra set di dati espressi con ordini di grandezza o unità di misura differenti, si ricorre al Mean Absolute Percentage Error (MAPE). Il MAPE si calcola come la media dei valori assoluti degli errori percentuali, i quali sono ottenuti rapportando ogni errore di previsione al rispettivo valore osservato. Formalmente:

$$MAPE = \frac{1}{H}\sum_{h=1}^{H}\left|\frac{e_h}{Y_h}\cdot 100\right| \tag{2.10}$$

Sebbene il MAPE abbia il vantaggio di essere privo di unità di misura, presenta criticità operative rilevanti: diventa infinito o indefinito se una qualsiasi osservazione nel periodo di interesse è pari a zero e può assumere valori estremi se le osservazioni sono molto vicine

allo zero. Inoltre, il MAPE ha lo svantaggio di applicare una penalità più pesante agli errori negativi rispetto a quelli positivi.

In conclusione, la scelta della misura di accuratezza ottimale dipende dal contesto applicativo e dalla distribuzione dei dati: il MAE è ideale per una valutazione robusta sulla mediana in serie con scale omogenee, l'RMSE è il riferimento per la precisione sulla media e la sensibilità agli outlier, mentre il MAPE resta lo standard per confronti tra serie eterogenee, a condizione che i dati non presentino valori nulli o prossimi allo zero, che ne renderebbero instabile il calcolo.

3. Le Extreme Learning Machines

3.1 Le Extreme Learning Machines per la stima delle reti neurali artificiali

Le Extreme Learning Machines (ELM) appartengono a una classe di reti neurali feed-forward in cui i parametri dello strato nascosto (o hidden layer) vengono generati in modo aleatorio e mantenuti costanti, mentre i pesi di output sono stimati per via analitica mediante procedure basate sui minimi quadrati regolarizzati. Sotto il profilo statistico, questa tipologia di reti viene denominata *Randomly Weighted Neural Networks* (RWNN) e non si configura come un nuovo paradigma di modellizzazione, bensì come una strategia di addestramento rapida e flessibile applicabile ai modelli classici di reti neurali. La forma funzionale delle RWNN coincide con quella dei tipici modelli di regressione non parametrica, fondati su combinazioni lineari di funzioni di base non lineari, permettendone un'interpretazione in termini di stimatori *sieve* o di approssimazioni a base stocastica.

All'interno del quadro generale delle reti neurali a pesi casuali, un ruolo di particolare rilevanza è rivestito dai modelli Extreme Learning Machine (ELM), originariamente proposti da Huang (Huang et al., 2004) [5], negli ultimi decenni hanno dato origine a una vasta letteratura.

Al fine di descrivere formalmente l'architettura ELM, risulta opportuno partire dalla sua rappresentazione funzionale; le ELM sono reti neurali caratterizzate da pesi casuali e prive di bias di output (o termine noto di output). Esse sono definite come segue:

$$\begin{aligned} f(\mathbf{x}, \boldsymbol{\theta}) &= \sum_{j=1}^{s} \beta_j \phi_j(\mathbf{x}) \\ &= \sum_{j=1}^{s} \beta_j \psi(\mathbf{x}'\mathbf{a}_j + b_j) \end{aligned} \tag{3.1}$$

dove $\boldsymbol{\theta}' = (\beta_1, \ldots, \beta_s, \mathbf{a}'_1, \ldots, \mathbf{a}'_s, b_1, \ldots, b_s)$. La topologia della rete è illustrata nella rappresentazione grafica in Figura 3.1, ed è composta da: un livello di input, un livello nascosto di funzioni di base e un singolo nodo di output. Le connessioni casuali del livello nascosto sono rappresentate in grigio chiaro, mentre i pesi delle connessioni tra il livello nascosto e il livello di output sono tracciati con linee più scure.

Analogamente ad altre reti neurali ottenute assegnando dei pesi casuali, l'addestramento delle ELM segue una procedura semplice e strutturata, la quale può essere scomposta in due fasi principali.

La prima fase concerne la costruzione dello strato nascosto, la quale viene eseguita in modo indipendente dai dati; in questo passaggio, l'ELM inizializza casualmente il livello nascosto al fine di mappare i dati di input nello spazio delle features avvalendosi di alcune funzioni di attivazione non lineari. Come nella teoria classica delle reti neurali, queste possono essere generiche funzioni non lineari continue a tratti, come le funzioni sigmoidali o ReLU, trattate all'interno del paragrafo 1.5.

I parametri dei nodi nascosti $\{(\mathbf{a}'_j, b_j), j = 1, \ldots, s\}$ sono generati in modo aleatorio secondo opportune distribuzioni di probabilità continue. Tra le scelte più comuni si annoverano $\mathbf{a}_j \sim \mathcal{U}(-1, 1)^d$ (distribuzione uniforme multivariata di dimensione d) oppure $\mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_d)$ (distribuzione Normale multivariata di dimensione d) per i pesi di input, e $b_j \sim \mathcal{U}(-1, 1)$ oppure $\mathcal{N}(0, \sigma^2)$ per i bias.

Nella pratica, tuttavia, la scala effettiva dei pesi e dei bias generati casualmente interagisce sia con il campo di variazione dei dati di input, sia con la scelta della funzione di attivazione.

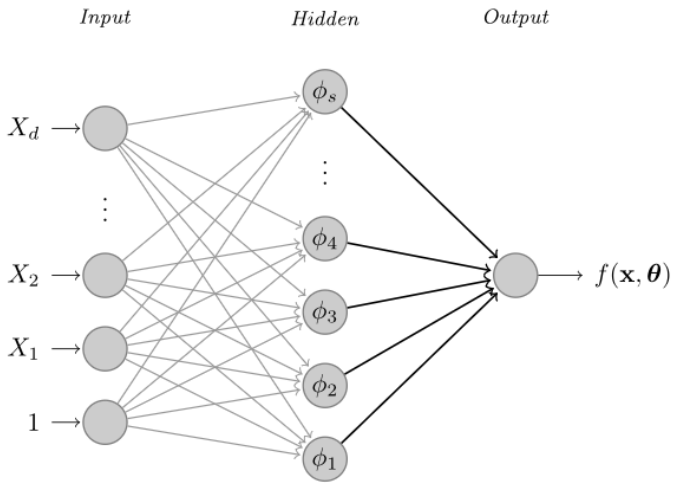


Figura 3.1: Rappresentazione della topologia di una Extreme Learning Machine (ELM)

Al fine di mitigare tale dipendenza e garantire rappresentazioni stabili delle features, è prassi consolidata standardizzare le variabili di input prima dell'addestramento, come consuetudine nelle applicazioni delle reti neurali; oltre alla standardizzazione, diversi contributi hanno dimostrato che vincolare o riscalare in modo appropriato i pesi e i bias casuali può migliorare le prestazioni di generalizzazione e di approssimazione (Wang et al., 2012 [4]).

Dato il campione casuale $\{(Y_i, \mathbf{x}_i'), i = 1, 2, \ldots, N\}$, il modello statistico basato su ELM può essere definito come:

$$Y_i = \sum_{j=1}^{s} \beta_j \psi(\mathbf{x}_i' \mathbf{a}_j + b_j) + \varepsilon_i, \quad i = 1, 2, \ldots, N \tag{3.2}$$

dove $\varepsilon_i \overset{iid}{\sim} (0, \sigma^2)$.

Il modello ELM per il campione osservato può essere espresso in forma matriciale come:

$$\mathbf{y} = \mathbf{H}\boldsymbol{\beta} + \mathbf{e} \tag{3.3}$$

dove $\mathbf{y}' = (Y_1, \ldots, Y_N)$ è il vettore di ordine N contenente le osservazioni sulla variabile dipendente, $\boldsymbol{\beta}' = (\beta_1, \ldots, \beta_s)$ è il vettore di ordine s contenente i pesi delle connessioni tra il livello nascosto e il livello di output, $\mathbf{e}' = (e_1, e_2, \ldots, e_N)$ è un vettore di ordine N contenente le realizzazioni dei termini di errore per il campione osservato, e $\mathbf{H}$ è una matrice di ordine $N \times s$ definita come:

$$\mathbf{H} = \begin{bmatrix} \psi(\mathbf{x}_1' \mathbf{a}_1 + b_1) & \ldots & \psi(\mathbf{x}_1' \mathbf{a}_s + b_s) \\ \vdots & \ddots & \vdots \\ \psi(\mathbf{x}_N' \mathbf{a}_1 + b_1) & \ldots & \psi(\mathbf{x}_N' \mathbf{a}_s + b_s) \end{bmatrix} \tag{3.4}$$

la quale è completamente nota.

Nella seconda fase, una volta fissata la trasformazione del livello nascosto, l'apprendimento si riduce alla stima dei pesi del livello di output $\boldsymbol{\beta}$, ottenuta risolvendo il seguente problema di minimizzazione:

$$\hat{\boldsymbol{\beta}} = \arg\min_{\boldsymbol{\beta}} ||\mathbf{H}\boldsymbol{\beta} - \mathbf{y}||^2 \tag{3.5}$$

dove $||\cdot||$ denota la norma L_2. Se $\mathbf{H}^\dagger$ indica l'inversa generalizzata di Moore-Penrose della matrice $\mathbf{H}$, per cui la soluzione ottimale al problema di ottimizzazione precedente risulta essere:

$$\hat{\boldsymbol{\beta}} = \mathbf{H}^\dagger \mathbf{y} \tag{3.6}$$

Al fine di migliorare la stabilità numerica e le prestazioni di generalizzazione, analogamente a quanto avviene nell'addestramento delle reti neurali standard, è possibile considerare una funzione obiettivo che includa sia l'errore di addestramento sia la norma L_2 dei pesi di output:

$$\hat{\boldsymbol{\beta}} = \arg\min_{\boldsymbol{\beta}} ||\mathbf{H}\boldsymbol{\beta} - \mathbf{y}||^2 + \lambda||\boldsymbol{\beta}||^2 \tag{3.7}$$

L'introduzione del termine di penalizzazione $\lambda||\boldsymbol{\beta}||^2$ è coerente con la teoria secondo cui i pesi di output $\boldsymbol{\beta}$ di entità ridotta svolgono un ruolo importante affinché i modelli ELM raggiungano una migliore generalizzazione.

A seconda delle dimensioni relative del campione e del livello nascosto, si ottengono soluzioni diverse in forma chiusa. Se $\mathbf{H}$ presenta più righe che colonne ($N > s$), come tipicamente si verifica nella pratica, vale la seguente espressione:

$$\hat{\boldsymbol{\beta}} = \left(\mathbf{H}'\mathbf{H} + \lambda \mathbf{I}_s\right)^{-1} \mathbf{H}'\mathbf{y} \tag{3.8}$$

dove $\mathbf{I}_s$ è una matrice identità di dimensione s e λ è un'opportuna costante predefinita.

Viceversa, quando il numero di osservazioni di addestramento risulta inferiore al numero di neuroni nascosti ($N < s$), i pesi di output possono essere stimati come:

$$\hat{\boldsymbol{\beta}} = \mathbf{H}' \left(\mathbf{H}\mathbf{H}' + \lambda \mathbf{I}_N\right)^{-1} \mathbf{y} \tag{3.9}$$

dove, in questo caso, $\mathbf{I}_N$ è una matrice identità di dimensione N. In entrambi i casi, l'aggiunta di una costante positiva alla diagonale della rispettiva matrice delle equazioni normali corrisponde a una regolarizzazione L_2 dei pesi di output.

Da una prospettiva numerica, questa operazione migliora il condizionamento della matrice da invertire, riducendo la sensibilità alla quasi-collinearità tra le features casuali. Da un punto di vista statistico, lo shrinkage dei pesi di output controlla la varianza e contribuisce a prevenire l'overfitting, migliorando così tipicamente le prestazioni di generalizzazione, specialmente in campioni finiti e in contesti ad alta dimensionalità (Deng et al., 2009 [5]).

La scelta del parametro di regolarizzazione λ gioca un ruolo cruciale nel bilanciare la distorsione e la varianza; mentre valori elevati di λ inducono una contrazione più forte e una distorsione potenzialmente maggiore, valori ridotti potrebbero non stabilizzare adeguatamente la stima.

Poiché i modelli ELM riducono l'addestramento alla stima di un modello lineare all'interno di uno spazio delle feature casuali fisso, è possibile sfruttare direttamente l'estesa letteratura sui modelli lineari regolarizzati. In tale framework, gli schemi di penalizzazione classici e moderni sviluppati per la regressione lineare possono essere applicati al livello di output per indurre shrinkage, sparsity o una regolarizzazione strutturata.

Al di là della loro attrattiva computazionale, le ELM poggiano altresì su solide basi teoriche. Nonostante la selezione casuale dei parametri del livello nascosto, questa classe di modelli conserva la proprietà di approssimazione universale, e risultati di consistenza, oltre che di approssimazione, sono stati stabiliti per contesti sia di regressione sia di classificazione. Un'analisi completa e una valutazione critica della fattibilità teorica delle ELM, unitamente a una discussione dei loro vantaggi e limiti, è fornita da Liu et al. (2015)[6] e Lin et al. (2015)[7]; tali lavori esaminano le condizioni sotto le quali le ELM raggiungono l'approssimazione universale, analizzano gli effetti della parametrizzazione casuale del livello

nascosto sull'accuratezza dell'approssimazione e sulla stabilità numerica, e chiariscono il trade-off tra efficienza computazionale e prestazioni statistiche.

Da un punto di vista pratico, la costruzione casuale dello strato nascosto comporta implicazioni non trascurabili in termini di complessità e di efficienza del modello. Invero, in ragione della generazione aleatoria dello strato nascosto, le ELM richiedono tipicamente un numero di neuroni nascosti superiore rispetto a quello necessario alle reti neurali completamente ottimizzate per il conseguimento di una determinata performance. Tale apparente svantaggio risulta tuttavia ampiamente compensato dalla notevole efficienza computazionale che contraddistingue le ELM, il cui tempo di addestramento può risultare inferiore di diversi ordini di grandezza rispetto a quello richiesto dalle reti neurali addestrate mediante procedure di ottimizzazione integralmente iterative, giacché la stima dei parametri si riconduce alla risoluzione di un sistema lineare regolarizzato.

L'efficienza computazionale delle ELM assume un ruolo determinante in ambito statistico, segnatamente nei contesti che richiedono l'implementazione di stime iterative, quali la cross-validation e il ricampionamento bootstrap. Sfruttando la rapidità di addestramento di tali reti, risulta possibile implementare procedure di *neural network sieve-bootstrap* (La Rocca e Perna, 2019 [8]), le quali garantiscono un'inferenza affidabile e una rigorosa quantificazione dell'incertezza in scenari non lineari. Un simile approccio consente di preservare la trattabilità computazionale anche in presenza di un numero elevato di replicazioni. L'architettura delle ELM si presta, inoltre, in modo intrinseco all'elaborazione di dataset ad alta dimensionalità, favorendo l'integrazione con paradigmi di calcolo parallelo e distribuito ai fini di un'ulteriore ottimizzazione dei tempi di convergenza.

Sebbene la formulazione standard delle ELM preveda l'assegnazione casuale e indipendente dei pesi dello strato nascosto, la letteratura ha esplorato diverse strategie volte all'ottimizzazione di tale meccanismo di randomizzazione. Tra gli sviluppi principali si annoverano criteri di inizializzazione finalizzati alla preservazione della topologia dei dati, nonché procedure costruttive in grado di determinare dinamicamente la cardinalità ottimale dei nodi nascosti, mantenendo al contempo inalterata la stima analitica in forma chiusa dei pesi di output.

L'impianto originario è stato, in seguito, oggetto di estensione secondo molteplici direzioni metodologiche. Un'evoluzione di rilievo è rappresentata dalle kernel ELM, le quali sostituiscono la mappatura casuale esplicita con una rappresentazione fondata su funzioni kernel. Tale formulazione coniuga l'efficienza algoritmica propria delle ELM con la flessibilità dei metodi kernel, consentendo di gestire in modo implicito proiezioni non lineari dello spazio delle feature. Parallelamente, al fine di incrementare la stabilità numerica e le performance predittive, sono state introdotte varianti incrementali, nelle quali i neuroni nascosti vengono integrati sequenzialmente nel corso del processo di apprendimento.

Infine, con l'obiettivo di mitigare l'impatto di valori anomali e di *heavy-tailed noise*, sono state sviluppate declinazioni robuste del modello. Tali approcci sostituiscono la tradizionale funzione di perdita quadratica con metriche dotate di maggiore resilienza, quali la funzione di perdita di Huber ovvero formulazioni ibride (e.g., Pinball-Huber). L'impiego di tali funzioni di costo, sovente abbinato a procedure di minimi quadrati ponderati iterativi, conferisce al framework una spiccata stabilità parametrica, rendendolo particolarmente idoneo a contesti di regressione complessi.

3.2 Previsione delle serie storiche con le Extreme Learning Machine

Questa sezione si sviluppa partendo direttamente dalla trattazione generale delle reti neurali per la previsione delle serie storiche, specializzandola per il caso delle reti ELM.

Data una serie storica $\{Y_t, t = 1, 2, \ldots, T\}$, il modello ELM-NAR($p$) è definito come:

$$Y_t = \sum_{j=1}^{s} \beta_j \psi\left(\mathbf{a}_j' \mathbf{x}_{t-1} + b_j\right) + \varepsilon_t, \quad t = p+1, \ldots, T \tag{3.10}$$

dove $\mathbf{x}_{t-1}' = (Y_{t-1}, Y_{t-2}, \ldots, Y_{t-p})$ raccoglie le p osservazioni ritardate. In forma vettoriale, il modello può essere scritto come segue:

$$Y_t = \psi\left(\mathbf{A}'\mathbf{x}_{t-1} + \mathbf{b}\right)' \boldsymbol{\beta} + \varepsilon_t, \quad t = p+1, \ldots, T \tag{3.11}$$

dove la funzione $\psi(\cdot)$ agisce per componenti, $\mathbf{b}' = (b_1, \ldots, b_s)$ e $\boldsymbol{\beta}' = (\beta_1, \beta_2, \ldots, \beta_s)$ sono vettori di dimensione s, e $\mathbf{A}$ è una matrice di dimensione $p \times s$ definita come:

$$\mathbf{A} = [\mathbf{a}_1, \mathbf{a}_2, \ldots, \mathbf{a}_s] = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1s} \\ \vdots & \vdots & \ddots & \vdots \\ a_{p1} & a_{p2} & \cdots & a_{ps} \end{bmatrix}. \tag{3.12}$$

Dati i valori osservati e impilando le osservazioni nel tempo, l'equazione 3.11 può essere espressa in forma matriciale come:

$$\mathbf{y} = \mathbf{H}\boldsymbol{\beta} + \mathbf{e} \tag{3.13}$$

dove $\mathbf{y}' = (Y_T, Y_{T-1}, \ldots, Y_{p+1})$ ed $\mathbf{e}' = (e_T, e_{T-1}, \ldots, e_{p+1})$ sono vettori di dimensione $T-p$, con $\mathbf{e}$ che include le realizzazioni dei termini di errore. La matrice del livello nascosto $\mathbf{H}$, di dimensione $(T-p) \times s$, è definita come:

$$\mathbf{H} = \psi(\mathbf{X}\mathbf{A} + \mathbf{1}_{T-p}\mathbf{b}') = \begin{bmatrix} \psi(\mathbf{a}_1'\mathbf{x}_{T-1} + b_1) & \cdots & \psi(\mathbf{a}_s'\mathbf{x}_{T-1} + b_s) \\ \vdots & \ddots & \vdots \\ \psi(\mathbf{a}_1'\mathbf{x}_p + b_1) & \cdots & \psi(\mathbf{a}_s'\mathbf{x}_p + b_s) \end{bmatrix} \tag{3.14}$$

dove $\mathbf{1}_{T-p}$ è un vettore di uni e la matrice $\mathbf{X}$ raccoglie i vettori di input ritardati:

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_{T-1}' \\ \mathbf{x}_{T-2}' \\ \vdots \\ \mathbf{x}_p' \end{bmatrix} = \begin{bmatrix} Y_{T-1} & \cdots & Y_{T-p} \\ Y_{T-2} & \cdots & Y_{T-p-1} \\ \vdots & \ddots & \vdots \\ Y_p & \cdots & Y_1 \end{bmatrix}. \tag{3.15}$$

Una volta che i parametri del livello nascosto $\mathbf{a}_1', \ldots, \mathbf{a}_s', b_1, \ldots, b_s$ vengono generati in modo casuale e fissati, la matrice $\mathbf{H}$ risulta completamente determinata e lo stimatore ELM si ricava in forma chiusa come:

$$\hat{\boldsymbol{\beta}} = \mathbf{H}^{\dagger}\mathbf{y} \tag{3.16}$$

dove $\mathbf{H}^{\dagger}$ è l'inversa generalizzata di Moore-Penrose. Tutte le considerazioni discusse nella sezione precedente, incluse le stime regolarizzate, si applicano direttamente anche al contesto delle serie storiche.

La previsione procede seguendo le medesime linee guida del caso generale delle serie storiche con reti neurali introdotte nel capitolo 2. Adottando la strategia ricorsiva, il percorso di previsione a h passi in avanti è dato da:

$$\hat{Y}_T(h) = \psi\left(\mathbf{A}'\mathbf{x}_{T+h-1} + \mathbf{b}\right)' \hat{\boldsymbol{\beta}}, \quad h = 1, 2, \ldots, \mathcal{H} \tag{3.17}$$

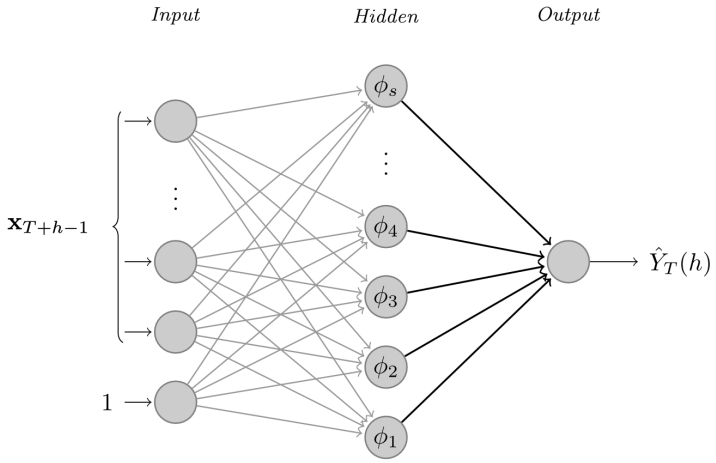


Figura 3.2: Rappresentazione della topologia di una rete ELM-NAR

dove il vettore di input $\mathbf{x}'_{T+h-1}$ è:

$$\mathbf{x}'_{T+h-1} = \begin{cases} (Y_T, \ldots, Y_{T-p+1}) & h = 1 \\ (\hat{Y}_T(h-1), \ldots, \hat{Y}_T(1), Y_T, \ldots, Y_{T-p+h}) & 2 \leq h \leq p \\ (\hat{Y}_T(h-1), \ldots, \hat{Y}_T(h-p)) & p+1 \leq h \leq \mathcal{H}. \end{cases} \tag{3.18}$$

La previsione ricorsiva a h passi in avanti basata sul modello ELM-NAR è illustrata nella Figura 3.2, che rappresenta un modello ELM-NAR con uno strato di input, uno strato nascosto di s nodi e un nodo di output che produce una singola previsione ricorsiva a h passi in avanti; in cui le connessioni casuali dello strato nascosto sono mostrate in grigio chiaro, mentre i pesi dallo strato nascosto all'output sono mostrati con linee più scure.

Come discusso in precedenza, la strategia ricorsiva è computazionalmente efficiente, ma può risentire dell'accumulo degli errori all'aumentare dell'orizzonte di previsione. Il metodo di previsione diretta stima invece $\mathcal{H}$ modelli ELM-NAR separati, uno per ciascun orizzonte di previsione. Nello specifico,

$$Y_t = \psi\left(\mathbf{A}'_h \mathbf{x}_{t-h} + \mathbf{b}_h\right)' \boldsymbol{\beta}_h + \varepsilon_{th}, \quad t = p+h, \ldots, T;\ h = 1, \ldots, \mathcal{H}, \tag{3.19}$$

e le previsioni sono conseguentemente calcolate come:

$$Y_T(h) = \psi\left(\mathbf{A}'_h \mathbf{x}_T + \mathbf{b}_h\right)' \hat{\boldsymbol{\beta}}_h, \quad h = 1, \ldots, \mathcal{H}. \tag{3.20}$$

Lo schema di previsione per uno specifico orizzonte h è illustrato nella Figura 3.3, che anche in questo caso è composto da uno strato di input, uno strato nascosto di s nodi e un nodo di output che produce una singola previsione diretta a h passi avanti.

L'approccio multi-input multi-output per i modelli ELM-NAR consegue in modo naturale e rispecchia il framework generale di previsione multi-output discusso in precedenza. Definendo il vettore dei ritardi $\mathbf{x}'_t = (Y_t, Y_{t-1}, \ldots, Y_{t-p+1})$, e il vettore dei valori futuri $\mathbf{y}'_{t:\mathcal{H}} = (Y_{t+1}, Y_{t+2}, \ldots, Y_{t+\mathcal{H}})$, la previsione viene formulata come un problema di regressione multi-output. I parametri dello strato nascosto $\mathbf{A}$ e $\mathbf{b}$ vengono generati casualmente e mantenuti fissi, e la matrice dello strato nascosto diventa

$$\mathbf{H} = \psi(\mathbf{X}\mathbf{A} + \mathbf{1}_n \mathbf{b}') \tag{3.21}$$

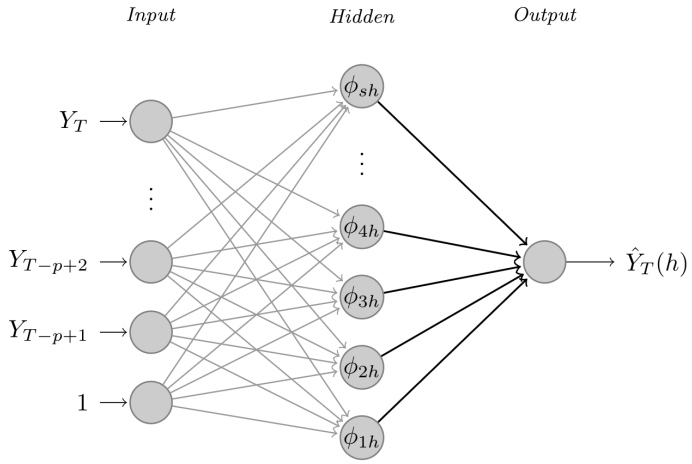


Figura 3.3: Rappresentazione della topologia di una rete ELM-NAR con previsione diretta a h step avanti

dove $\psi(\cdot)$ agisce per elementi e $\mathbf{1}_n$ è un vettore di uni, con $n = T - p - \mathcal{H} + 1$, e $\mathbf{X}$ è una matrice di ordine $n \times \mathcal{H}$ definita come:

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}'_{T-\mathcal{H}} \\ \mathbf{x}'_{T-\mathcal{H}-1} \\ \vdots \\ \mathbf{x}'_{p} \end{bmatrix} \tag{3.22}$$

Il modello ELM-NAR multi-output diventa:

$$\mathbf{Y} = \mathbf{H}\mathbf{B} + \mathbf{E} \tag{3.23}$$

dove $\mathbf{H}$ è la matrice di ordine $n \times s$ definita nell'equazione (3.13), $\mathbf{E}$ è la matrice di ordine $n \times \mathcal{H}$ contenente le realizzazioni dei termini di errore, $\mathbf{B}$ è una matrice di ordine $s \times \mathcal{H}$ contenente tutti i pesi di output, e $\mathbf{Y}$ è una matrice di ordine $n \times \mathcal{H}$. Le matrici $\mathbf{B}$ e $\mathbf{Y}$ sono definite come:

$$\mathbf{B} = \begin{bmatrix} \beta_{11} & \beta_{12} & \cdots & \beta_{1\mathcal{H}} \\ \vdots & \vdots & \ddots & \vdots \\ \beta_{s1} & \beta_{s2} & \cdots & \beta_{s\mathcal{H}} \end{bmatrix}, \quad \mathbf{Y} = \begin{bmatrix} \mathbf{y}'_{T-\mathcal{H},\mathcal{H}} \\ \mathbf{y}'_{T-\mathcal{H}-1,\mathcal{H}} \\ \vdots \\ \mathbf{y}'_{p,\mathcal{H}} \end{bmatrix} \tag{3.24}$$

Utilizzando i minimi quadrati regolarizzati tramite il metodo ridge, i pesi di output stimati sono:

$$\hat{\mathbf{B}} = (\mathbf{H}'\mathbf{H} + \lambda \mathbb{I}_s)^{-1}\mathbf{H}'\mathbf{Y} \tag{3.25}$$

Dato l'ultimo vettore dei ritardi $\mathbf{x}'_T = (Y_T, Y_{T-1}, \ldots, Y_{T-p+1})$, le previsioni multi-input multi-output sono:

$$\hat{\mathbf{y}}_T(\mathcal{H}) = \begin{bmatrix} \hat{Y}_T(1) \\ \vdots \\ \hat{Y}_T(\mathcal{H}) \end{bmatrix} = \psi(\mathbf{A}'\mathbf{x}_T + \mathbf{b})'\hat{\mathbf{B}} \tag{3.26}$$

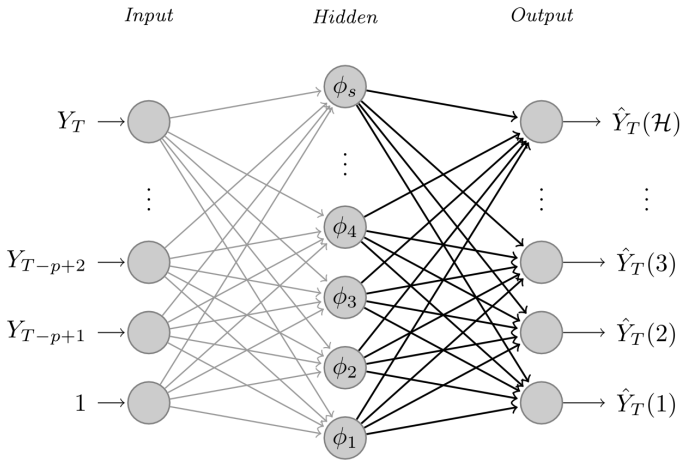


Figura 3.4: Rappresentazione del grafo di una rete ELM-NAR multi-input multi-output per la previsione multi-step

Ciascuna componente corrisponde a uno specifico orizzonte di previsione, mentre tutte le previsioni sono generate dalla medesima mappa delle feature casuale.

Lo schema di previsione multi-input multi-output è illustrato nella Figura 3.4.

Le reti multi-output possono essere facilmente adattate al caso della previsione di serie storiche non lineari multivariate. Sia $\mathbf{Y}_t$ una serie storica multivariata con m componenti $(Y_{t1}, Y_{t2}, \ldots, Y_{tm})$. Si definisca la mappa delle feature dello strato nascosto come $\psi(\mathbf{A}'\mathbf{x}_{t-1} + \mathbf{b})$, dove il vettore di input $\mathbf{x}'_{t-1} = (\mathbf{y}'_{t-1}, \mathbf{y}'_{t-2}, \ldots, \mathbf{y}'_{t-p})$ con $\mathbf{y}'_{t-j} = (Y_{t-j,1}, \ldots, Y_{t-j,m})$ vettore di ordine m, $\mathbf{A} = [\mathbf{a}_1 \ \ldots \ \mathbf{a}_s]$ è una matrice di ordine $mp \times s$ e $\mathbf{b} = (b_1, \ldots, b_s)'$ è un vettore di ordine s.

Un modello ELM-NAR multi-output per serie storiche multivariate può quindi essere scritto come:

$$\mathbf{y}_t = \psi(\mathbf{A}'\mathbf{x}_{t-1} + \mathbf{b})'\mathbf{B} + \boldsymbol{\varepsilon}_t, \tag{3.27}$$

dove $\boldsymbol{\varepsilon}_t$ è un vettore di ordine m di termini di errore e $\mathbf{B}$ è la matrice di ordine $s \times m$ contenente i parametri da stimare.

Una volta che i parametri dello strato nascosto vengono generati casualmente e fissati, e $\mathbf{B}$ è stimata tramite i minimi quadrati (regolarizzati), le previsioni a un passo in avanti per l'istante $T + 1$, date le informazioni disponibili al tempo T, per le m componenti vettoriali, sono date da:

$$\hat{\mathbf{y}}_T(1) = \psi(\mathbf{A}'\mathbf{x}_T + \mathbf{b})'\hat{\mathbf{B}}. \tag{3.28}$$

Tale costruzione si estende in modo immediato a insiemi di ritardi generali (e persino a ritardi differenti tra le componenti) ridefinendo $\mathbf{x}_t$ in modo da includere i ritardi prescelti; unicamente la dimensione di input subisce variazioni. Le estensioni alle previsioni a più passi in avanti (multistep-ahead) sono dirette ma algebricamente tediose; pertanto, non vengono qui riportate per ragioni di semplicità.

Gli intervalli di previsione possono essere costruiti mediante l'impiego di tecniche di bootstrap, sfruttando l'elevata efficienza computazionale che caratterizza l'addestramento delle Extreme Learning Machine. In particolare, la possibilità di stimare in forma chiusa i pesi di output consente di ristimare ripetutamente i modelli ELM su serie ricampionate, ovvero

generate mediante procedure di bootstrap, a fronte di un costo computazionale estremamente contenuto. Tale caratteristica rende concretamente praticabile l'approssimazione dell'intera distribuzione predittiva attraverso la generazione di un numero elevato di previsioni bootstrap, dalle quali è possibile derivare direttamente gli intervalli di previsione empirici. Un approccio di questo tipo risulta di particolare interesse nell'ambito delle serie storiche non lineari, contesto in cui le espressioni analitiche per la quantificazione dell'incertezza previsiva risultano generalmente non disponibili. Come dimostrato da La Rocca e Perna (2023) [9], la combinazione delle ELM con schemi di ricampionamento offre un framework flessibile e computazionalmente efficiente per la quantificazione dell'incertezza previsiva, preservando al contempo la struttura non lineare del processo generatore dei dati sottostante.

Le distribuzioni di previsione ottenute mediante modelli fondati su reti neurali offrono una rappresentazione ricca delle dinamiche sottostanti alle serie storiche non lineari e possono, di conseguenza, essere proficuamente impiegate a fini di clustering. Anziché fondarsi su previsioni puntuali o su statistiche di sintesi, tale approccio pone a confronto intere distribuzioni predittive, le quali sono in grado di catturare sia la tendenza centrale sia l'incertezza associata alle osservazioni future. Attraverso la costruzione di distribuzioni di previsione per mezzo di tecniche di bootstrap o di ricampionamento, i modelli a reti neurali risultano capaci di riflettere le caratteristiche non lineari e la dipendenza temporale presenti nei dati. Il clustering delle serie storiche, condotto sulla base delle similarità riscontrate tra tali distribuzioni predittive, dà luogo a raggruppamenti significativi, robusti rispetto al rumore e all'eterogeneità dei dati, e risulta particolarmente idoneo all'analisi di serie storiche economiche complesse, laddove i metodi di clustering tradizionali, fondati su misure di distanza, possono rivelarsi inadeguati.

4. Caso studio: analisi comparativa dei tempi di addestramento con dati simulati

4.1 Modelli e metodologia utilizzati

Il percorso seguito per la determinazione della metodologia da utilizzare si basa sul lavoro di La Rocca e Perna (2023) [9], già citato in precedenza; in particolare, nel corso della quarta sezione del lavoro sono stati valutati i vantaggi in termini di performance computazionali dei modelli ELM rispetto alle reti neurali feedforward classiche. Il confronto è stato effettuato utilizzando 5000 serie storiche di diversa lunghezza generate da un processo EXPAR con un numero di ritardi pari a 2; prima di approfondire il disegno sperimentale, forniamo una definizione formale.

4.1.1 Modello EXPAR

Il modello Exponential Autoregressive (EXPAR) rappresenta una specifica classe di modelli parametrici per serie storiche non lineari, ampiamente utilizzati per descrivere dinamiche stocastiche che non possono essere catturate dai modelli lineari tradizionali. Esso si colloca all'interno della famiglia dei processi Nonlinear Autoregressive (NAR) di ordine p; la caratteristica distintiva del modello EXPAR è che i suoi coefficienti autoregressivi variano in modo esponenziale in funzione dei valori passati della serie stessa, permettendo al modello di cambiare comportamento a seconda dell'ampiezza delle osservazioni precedenti.

Un esempio tipico di modello EXPAR(1) è definito dalla seguente equazione:

$$Y_t = [\alpha_{1,1} + \alpha_{1,2} \exp(-Y_{t-1}^2)]Y_{t-1} + \epsilon_t \tag{4.1}$$

dove i parametri determinano la velocità di transizione tra diversi stati. Nel caso di un modello di ordine superiore, come l'EXPAR(2), la struttura assume la forma seguente:

$$Y_t = a_1(Y_{t-1})Y_{t-1} + a_2(Y_{t-1})Y_{t-2} + \epsilon_t \tag{4.2}$$

dove:

$$\begin{aligned} a_1(u) &= \alpha_{1,1} + \alpha_{1,2} \exp(-u^2) \\ a_2(u) &= \alpha_{2,1} + \alpha_{2,2} \exp(-u^2) \end{aligned} \tag{4.3}$$

Le caratteristiche fondamentali che rendono il modello EXPAR di particolare interesse nella letteratura statistica includono:

- Transizione Continua: a differenza dei modelli a soglia (come i SETAR), che presentano bruschi cambiamenti di regime, l'EXPAR modella una transizione fluida e continua tra diversi stati della dinamica del sistema.
- Modellazione di Fenomeni Complessi: questa famiglia di modelli è particolarmente efficace nello spiegare:
  - Jump phenomena: improvvisi cambiamenti di livello o salti nella serie.
  - Amplitude-dependent frequency shifts: variazioni della frequenza delle oscillazioni che dipendono dall'ampiezza del segnale.
  - Perturbed limit cycles: cicli limite perturbati che descrivono sistemi che tendono a tornare a un'oscillazione stabile dopo una perturbazione.
- Flessibilità Parametrica: sebbene sia un modello parametrico, la sua capacità di approssimare dinamiche non lineari lo rende un termine di confronto ideale per valutare stimatori non parametrici globali, come le reti neurali artificiali (ANN).

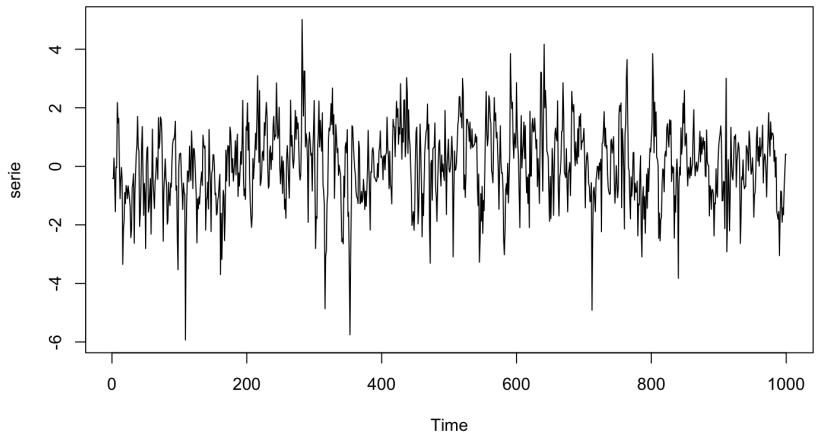


Figura 4.1: Rappresentazione grafica di una realizzazione di una serie simulata da un processo EXPAR(2) con numerosità campionaria $T = 1000$.

4.1.2 Impostazione del disegno sperimentale

Ricalcando quanto effettuato nel paper originale, sono state generate 5000 serie storiche di diversa lunghezza nell'insieme $\{250, 500, 1000\}$ a partire da un modello EXPAR(1) e altrettante serie da un modello EXPAR(2) con le forme funzionali viste rispettivamente nelle equazioni (4.1) e (4.2). I valori assunti nello specifico da a_1 e a_2 sono:

$$\begin{aligned} a_1(u) &= 0.5 + 0.9\exp(-u^2) \\ a_2(u) &= -0.8 + 1.8\exp(-u^2) \end{aligned} \tag{4.4}$$

con $\epsilon_t \sim \frac{T_6}{\sqrt{1.5}}$; dove T_6 è una T di student con 6 gradi di libertà. Si noti che, per il modello EXPAR(1), il parametro $a_2(u)$ è posto pari a zero. Si mostra in Figura 4.1, a titolo esemplificativo, il grafico di una realizzazione di una serie simulata da un processo EXPAR(2) con $T = 1000$.

Per ciascuna serie storica sono stati stimati un modello ELM e una rete neurale feedforward, dove lo strato di input è fissato a 1 e 2 neuroni rispettivamente per ricalcare la struttura di input del modello sottostante da cui sono stati generati i dati, mentre la grandezza dello strato nascosto varia nell'insieme di valori $\{4, 8, 12, 16, 20\}$, in modo da considerare modelli con complessità crescente che richiedono, a loro volta, un tempo di addestramento più elevato.

La rete neurale feedforward è stata stimata in R mediante l'utilizzo del pacchetto *nnet*, scritto in linguaggio C, mentre il codice per la stima dell'ELM è stato sviluppato nell'ambito del presente lavoro di tesi.

Tutte le reti neurali classiche sono state stimate riavviando l'addestramento per dieci volte, al fine di evitare che il processo rimanesse intrappolato in minimi locali (accorgimento non necessario nel caso in cui si utilizzino le ELM).

I risultati sono stati ottenuti calcolando la media dei tempi di computazione delle reti neurali classiche e la media dei tempi di computazione dei modelli ELM per ciascuna serie storica, tutti calcolati in secondi, costruendo il rapporto di tali tempi in-sample. In particolare, tra i dieci riavvii, il tempo scelto per calcolare la media è quello registrato dal modello migliore in termini di RMSE.

Rispetto al lavoro di La Rocca e Perna (2023), l'analisi condotta viene ampliata considerando anche la capacità di adattamento dei modelli stimati in-sample e out-of-sample; in aggiunta, verrà analizzato in che modo sia necessario variare la numerosità dello strato nascosto affinché le performance di adattamento, prima, e previsione, poi, possano essere assimilabili tra i due modelli. Pertanto, il vantaggio computazionale verrà valutato anche in relazione alle performance di adattamento e alle performance previsive ottenute dai modelli.

L'analisi relativa alla variazione della numerosità dello strato nascosto viene svolta attraverso una procedura iterativa di stima dei modelli ELM, i quali vengono stimati con un numero di neuroni m che cresce in modo incrementale di 4 unità per ogni iterazione, fino ad un tetto massimo di 400 neuroni nello strato intermedio. La procedura iterativa si blocca nel momento in cui l'RMSE ottenuto dal modello ELM stimato si avvicina all'RMSE del modello NNET, in particolare, ciò accade se l'RMSE calcolato è al più uguale all'errore del modello NNET a cui viene aggiunto un 5% di tolleranza. Nelle tabelle che seguiranno il valore 400 non sarà quindi da intendersi come il valore effettivo necessario al modello ELM per ottenere performance similari ma semplicemente l'effetto di una mancata convergenza tra le performance a causa della combinazione di numerosità campionaria e numerosità dello strato nascosto.

4.2 Risultati

4.2.1 Reti neurali stimate su un processo EXPAR(1)

L'analisi dei risultati ottenuti inizia da quanto accade per il modello EXPAR(1). Una volta generate 5000 serie storiche provenienti dal modello teorico indicato nell'equazione (4.1) con i parametri impostati come da equazione (4.4) e stimati i relativi modelli seguendo l'impostazione definita nella sezione 4.1.2, risulta possibile calcolare i valori medi dei tempi di addestramento, l'RMSE medio e i rapporti nei tempi di addestramento sia in-sample che out-of-sample.

Risultati ottenuti in-sample

Per quanto riguarda i risultati ottenuti in-sample, riportati nella tabella 4.1 e graficamente nella figura 4.1, è possibile notare come: all'aumentare della lunghezza della serie, i due modelli risultino avere un RMSE medio molto vicino (per T=1000 ricadono entro il 5%); all'aumentare del numero di neuroni nello strato nascosto il vantaggio computazionale in termini di rapporto tra tempi di computazione aumenti fissando il numero di osservazioni, e come ciò valga anche all'aumentare del numero di osservazioni fissando il numero di neuroni nello strato nascosto. In linea con quanto riportato da La Rocca e Perna (2023), il rapporto nei tempi di computazione è positivo nell'ordine dalle 87 alle 376 volte, in base a complessità dello strato nascosto e numerosità della serie storica di riferimento; ulteriore conferma del carattere di crescita di tale rapporto è il modo in cui questi due valori estremi rappresentino rispettivamente il modello più semplice e il modello più complesso tra quelli stimati.

Oltre a tale analisi, data la natura di assegnazione casuale dei pesi dell'ELM, si è voluto analizzare se fosse possibile ottenere performance in-sample simili (entro il 5%) in termini di RMSE medio aumentando il numero di neuroni nello strato nascosto dei modelli ELM. L'analisi, i cui risultati sono riportati nella tabella 4.2, evidenzia come per alcune combinazioni di lunghezza della serie storica e composizione dello strato nascosto tale convergenza per il livello di errore non sia possibile anche considerando un numero arbitrariamente alto di neuroni nello strato nascosto (400). In questo contesto, la discriminante principale sembra essere proprio la lunghezza della serie storica, in quanto per $T = 1000$, come già dimostrato

Tabella 4.1: Confronto delle prestazioni in-sample (RMSE e rapporto dei tempi di addestramento) tra reti neurali feedforward (NNET) ed Extreme Learning Machines (ELM) su dati simulati da un processo EXPAR(1), al variare della numerosità campionaria (T) e dei neuroni nello strato nascosto (m)

T	m	Rapporto Tempi NNET/ELM	RMSE medio NNET	RMSE medio ELM
250	4	87.18	0.955	1.007
250	8	161.33	0.919	1.000
250	12	160.75	0.894	0.997
250	16	183.90	0.875	0.996
250	20	230.49	0.860	0.994
500	4	133.32	0.978	1.010
500	8	227.01	0.958	1.003
500	12	260.14	0.946	0.999
500	16	223.24	0.937	0.998
500	20	309.86	0.931	0.997
1000	4	191.19	0.990	1.011
1000	8	292.32	0.980	1.004
1000	12	274.39	0.974	1.002
1000	16	250.17	0.970	1.000
1000	20	376.25	0.967	0.999

in precedenza, i valori medi dell'RMSE delle due classi di modelli si avvicinano con un numero di neuroni nello strato nascosto che talvolta risulta essere anche inferiore rispetto al numero di neuroni utilizzati dall'omologo modello classico. Quando ciò accade, la magnitudine del rapporto analizzato risulta ancora più a favore per l'ELM che arriva ad essere fino a circa 1350 volte più veloce, a parità di capacità di adattamento ai dati in-sample.

Risultati ottenuti out-of-sample

Per quanto riguarda i risultati ottenuti out-of-sample, riportati nella Tabella 4.3, è possibile focalizzarsi sull'andamento del rapporto nei tempi di computazione tra NNET ed ELM a parità di RMSE.

Il primo risultato che emerge è la sistematica superiorità out-of-sample dell'ELM rispetto alle coppie di neuroni nello strato nascosto e lunghezze delle serie storiche; in particolare, su 15 combinazioni di (T, m), ELM ottiene un RMSE inferiore a NNET in 14 casi su 15. L'unica eccezione è $T = 1000$, $m = 12$, dove NNET (0.993) batte di poco ELM (0.997). Questo suggerisce che, per approssimare la dinamica non lineare generata dal modello EXPAR(1), l'approccio ELM è complessivamente più efficace e affidabile di una rete allenata con backpropagation standard.

Un secondo dato interessante riguarda il comportamento al crescere di m: NNET mostra un netto peggioramento al crescere di m quando T è piccolo: per $T = 250$ l'RMSE sale da 0.991 ($m = 4$) a 1.183 ($m = 20$), un incremento relativo di circa il 19%. Questo è il tipico sintomo della *curse of dimensionality*: all'aumentare del numero di input, cresce il numero di parametri da stimare (pesi delle connessioni), e con un campione ridotto la rete tende a overfittare in-sample, perdendo capacità predittiva out-of-sample. Ulteriore conferma è data dall'analisi incrociata con la Tabella 4.1, in cui l'RMSE minore è in corrispondenza del

Tabella 4.2: Rapporto tra i tempi di addestramento in-sample a parità di capacità di adattamento (tolleranza RMSE del 5%) tra NNET ed ELM per il modello EXPAR(1).

T	m NNET	Rapporto Tempi NNET/ELM	m^* ELM	Rapporto Tempi NNET/ELM*
250	4	87.18	4	87.18
250	8	161.33	400	0.42
250	12	160.75	400	0.63
250	16	183.90	400	0.81
250	20	230.49	400	1.01
500	4	133.32	4	133.32
500	8	227.01	4	376.02
500	12	260.14	20	192.02
500	16	223.24	400	1.13
500	20	309.86	400	1.42
1000	4	191.19	4	191.19
1000	8	292.32	4	565.10
1000	12	274.39	4	838.01
1000	16	250.17	4	1080.35
1000	20	376.25	4	1346.85

Tabella 4.3: Prestazioni predittive out-of-sample e confronto dei tempi di addestramento tra NNET ed ELM (con m^* neuroni) a parità di errore di addestramento (tolleranza 5%) per il processo EXPAR(1).

T	m NNET	RMSE medio NNET	m^* ELM	RMSE medio ELM	Rapporto Tempi NNET/ELM*
250	4	0.991	4	0.970	105.56
250	8	1.109	4	0.989	271.96
250	12	1.135	4	0.976	396.00
250	16	1.172	4	0.986	510.28
250	20	1.183	4	0.970	636.44
500	4	0.987	4	0.979	135.44
500	8	0.967	4	0.947	373.95
500	12	1.021	4	0.943	552.28
500	16	1.000	4	0.937	709.23
500	20	1.020	4	0.952	878.33
1000	4	0.982	4	0.980	228.24
1000	8	0.980	4	0.959	670.85
1000	12	0.993	4	0.997	992.56
1000	16	0.974	4	0.949	1271.85
1000	20	0.987	4	0.956	1590.61

modello con $T = 250$ e $m = 20$. Al contrario, l'ELM resta stabile al variare di T (range 0.937–0.997), con valori inferiori a quelli visti in-sample, dimostrando una buona capacità di generalizzazione.

Il risultato, dal punto di vista dell'efficienza computazionale, è netto: l'ELM raggiunge un RMSE equivalente con solo 4 neuroni nascosti e in tempi da 100 a oltre 1500 volte inferiori, valori assimilabili a quanto visto nella tabella 4.2 in-sample quando si è verificata la vicinanza entro il valore impostato di tolleranza tra i valori degli RMSE. Dato che $m^* = 4$ è un limite inferiore della griglia, il vero m^* potrebbe essere inferiore; i rapporti sono quindi limiti inferiori del vero rapporto ottenibile con questo criterio. Di fatti, il 5% è ininfluente, e questo rafforza il risultato.

4.2.2 Reti neurali stimate su un processo EXPAR(2)

L'analisi dei risultati ottenuti inizia da quanto accade per il modello EXPAR(2). Una volta generate 5000 serie storiche provenienti dal modello teorico indicato nell'equazione (4.2) con i parametri impostati come da equazione (4.4) e stimati i relativi modelli seguendo l'impostazione definita nella sezione 4.1.2, risulta possibile calcolare i valori medi dei tempi di addestramento, l'RMSE medio e i rapporti nei tempi di addestramento sia in-sample che out-of-sample.

Risultati ottenuti in-sample

Per quanto riguarda i risultati ottenuti in-sample, riportati nella tabella 4.4, è possibile notare come: il rapporto nei tempi di computazione sia positivo nell'ordine dalle circa 110 alle circa 434 volte; in base alla complessità dello strato nascosto e alla numerosità della serie storica di riferimento; ulteriore conferma del carattere di crescita di tale rapporto è il modo in cui questi due valori estremi rappresentino rispettivamente il modello più semplice e il modello più complesso tra quelli stimati, esattamente come accadeva per il processo EXPAR(1). A differenza del caso precedente l'RMSE dei due modelli risulta essere molto più distante in termini assoluti; tale risultato è atteso in quanto il processo generatore dei dati è più complesso e il processo di ottimizzazione delle reti NNET costituiscono un vantaggio in termini di adattamento; anche in questo caso il livello minimo di RMSE (0.692) si riscontra in corrispondenza della coppia $T = 250$, $m = 20$ che potrebbe denotare un sostanziale overfitting ai dati di addestramento.

Oltre a tale analisi, data la natura di assegnazione casuale dei pesi dell'ELM, si è voluto analizzare se fosse possibile ottenere performance in-sample simili (entro il 5%) in termini di RMSE medio delle reti neurali classiche aumentando il numero di neuroni nello strato nascosto dei modelli ELM. L'analisi, i cui risultati sono riportati nella tabella 4.5, evidenzia come per alcune combinazioni di lunghezza della serie storica e composizione dello strato nascosto tale convergenza per il livello di errore non sia possibile anche considerando un numero arbitrariamente alto di neuroni nello strato nascosto (400). In questo contesto, come per l'EXPAR(1), ma in maniera meno marcata, la discriminante principale sembra essere proprio la lunghezza della serie storica, in quanto per $T = 1000$, i valori medi dell'RMSE delle due classi di modelli si avvicinano, con un numero di neuroni nello strato nascosto che non risulta mai inferiore al numero di neuroni utilizzati dall'omologo modello classico. Quando il numero di neuroni nello strato nascosto è diverso dal tetto massimo impostato dalla griglia di 400, la magnitudine del rapporto analizzato risulta comunque più a favore dell'ELM che arriva ad essere fino in un range tra 21 e 30 volte più veloce, a parità di capacità di adattamento ai dati in-sample.

Tabella 4.4: Confronto delle prestazioni in-sample (RMSE e rapporto dei tempi di addestramento) tra reti neurali feedforward (NNET) ed Extreme Learning Machines (ELM) su dati simulati da un processo EXPAR(2), al variare della numerosità campionaria (T) e della complessità strutturale (m).

T	m	Rapporto Tempi NNET/ELM	RMSE medio NNET	RMSE medio ELM
250	4	110.35	0.950	1.374
250	8	180.51	0.873	1.239
250	12	201.90	0.805	1.151
250	16	276.00	0.746	1.104
250	20	136.54	0.692	1.072
500	4	197.41	0.974	1.379
500	8	270.52	0.935	1.253
500	12	276.61	0.901	1.170
500	16	241.69	0.870	1.125
500	20	345.14	0.842	1.092
1000	4	297.00	0.987	1.386
1000	8	351.05	0.968	1.261
1000	12	326.00	0.951	1.183
1000	16	287.36	0.936	1.138
1000	20	433.90	0.922	1.106

Tabella 4.5: Rapporto tra i tempi di addestramento in-sample a parità di capacità di adattamento (tolleranza RMSE del 5%) tra NNET ed ELM per il modello EXPAR(2).

T	m NNET	Rapporto Tempi NNET/ELM	m^* ELM	Rapporto Tempi NNET/ELM*
250	4	110.35	44	26.39
250	8	180.51	400	0.64
250	12	201.90	400	0.93
250	16	276.00	400	1.23
250	20	136.54	400	1.55
500	4	197.41	44	21.58
500	8	270.52	152	5.46
500	12	276.61	400	1.00
500	16	241.69	400	1.30
500	20	345.14	400	1.60
1000	4	297.00	44	25.58
1000	8	351.05	72	30.25
1000	12	326.00	160	7.61
1000	16	287.36	400	1.65
1000	20	433.90	400	2.04

In questo contesto, l'aumento della complessità del processo generatore dei dati si rispecchia in un livello per lo strato nascosto "a parità" (entro il 5% di RMSE) sistematicamente maggiore dal livello visto nell'analoga valutazione sull'EXPAR(1) (Tabella 4.2).

Risultati ottenuti out-of-sample

Per quanto riguarda i risultati ottenuti out-of-sample, riportati nella Tabella 4.6, è possibile focalizzarsi sull'andamento dell'RMSE.

Tabella 4.6: Prestazioni predittive out-of-sample e confronto dei tempi di addestramento tra NNET ed ELM (con m^* neuroni) a parità di errore di addestramento (tolleranza 5%) per il processo EXPAR(2).

T	m NNET	RMSE medio NNET	m^* ELM	RMSE medio ELM	Rapporto Tempi NNET/ELM*
250	4	1.022	20	1.047	29.77
250	8	1.098	12	1.093	81.78
250	12	1.184	12	1.108	89.64
250	16	1.258	4	1.142	205.14
250	20	1.301	4	1.163	246.87
500	4	0.988	32	1.030	28.77
500	8	0.995	20	1.031	72.40
500	12	1.069	16	1.060	107.45
500	16	1.070	12	1.071	228.28
500	20	1.060	16	1.058	177.44
1000	4	0.968	32	1.001	35.32
1000	8	0.997	20	1.015	93.41
1000	12	0.955	20	0.968	132.92
1000	16	1.018	16	1.026	217.94
1000	20	1.017	20	1.013	280.74

Il primo risultato significativo riguarda l'evoluzione dell'RMSE del modello NNET; mentre nella Tabella 4.4 si mostra il calo dell'RMSE in-sample più marcato al crescere di m, si riscontra un comportamento diametralmente opposto out-of-sample: l'RMSE aumenta costantemente e in modo consistente al crescere di m, passando da 1.022 ($T = 250, m = 4$) a 1.301 ($T = 250, m = 20$), un peggioramento relativo di quasi il 27%, mentre per le altre numerosità campionarie tale peggioramento è relativamente inferiore, circa il 5-7%.

La colonna del rapporto mostra che il tempo di addestramento della NNET è da 28 a oltre 280 volte superiore rispetto a quello dell'ELM. Questo divario si allarga drasticamente all'aumentare dei nodi m della rete base. Per soddisfare il vincolo di "parità" (RMSE ELM al più pari a RMSE NNET maggiorato del 5%), il modello ELM necessita quasi sempre di una dimensionalità maggiore nello strato nascosto. Ad esempio, per $T = 500$ e $m = 4$, l'ELM richiede $m^* = 32$ nodi per eguagliare la precisione; nonostante la necessità di espandere la rete per compensare l'assegnazione casuale dei pesi di input, il rapporto dei tempi rimane nettamente a favore dell'ELM, dimostrando che l'inversione matriciale è ordini di grandezza più veloce dell'ottimizzazione iterativa. Inoltre, l'aumento della lunghezza della serie storica T non degrada il vantaggio competitivo dell'ELM. Le proporzioni di efficienza si mantengono

simili o addirittura migliorano sui campioni più estesi, confermando la solidità dell'algoritmo nella gestione di set di dati più lunghi, tipici nell'analisi delle serie storiche.

4.3 Conclusioni e sviluppi futuri

Il presente lavoro di tesi si è posto l'obiettivo di analizzare e confrontare, sotto un duplice profilo teorico ed empirico, le prestazioni delle classiche Reti Neurali Artificiali (addestrate tramite l'algoritmo iterativo di *backpropagation*) e delle *Extreme Learning Machines* (ELM) nella stima e nella previsione di serie storiche non lineari. Attraverso un disegno sperimentale basato su processi generatori di dati di tipo EXPAR(1) ed EXPAR(2), è stato possibile valutare i due approcci in funzione della numerosità campionaria e della complessità dimensionale dello strato di input (numero di ritardi).

L'evidenza empirica ha confermato la solidità delle premesse teoriche, delineando un marcato trade-off tra flessibilità adattiva ed efficienza computazionale. Sotto il profilo dei tempi di addestramento, l'approccio ELM ha dimostrato una superiorità netta e sistematica, risultando da decine a migliaia di volte più rapido rispetto alle reti tradizionali, grazie alla stima analitica in forma chiusa dei pesi di output. Tuttavia, l'analisi delle performance previsive ha rivelato che la gerarchia tra i modelli non è assoluta, ma dipende intimamente dalla struttura del processo stocastico e dalla disponibilità di dati. Nel caso del processo EXPAR(1), l'ELM ha mostrato una straordinaria capacità di generalizzazione out-of-sample, specialmente in condizioni di scarsità campionaria (ad esempio, per $T = 250$), riuscendo a mitigare l'overfitting e il problema della *curse of dimensionality*, che penalizzano severamente la NNET all'aumentare dei ritardi immessi nella rete. Al contrario, a fronte di dinamiche più complesse introdotte dal processo EXPAR(2), l'ottimizzazione integralmente iterativa della NNET ha garantito un migliore adattamento in-sample per campioni estesi, sebbene l'ELM abbia mantenuto performance previsive estremamente competitive. Per eguagliare i livelli di errore della NNET, il modello ELM ha richiesto un ampliamento della dimensionalità dello strato nascosto, conservando in ogni caso un vantaggio computazionale difficilmente colmabile.

I risultati conseguiti in questo studio offrono importanti indicazioni metodologiche e aprono la strada a diverse e promettenti direzioni di ricerca futura. In primo luogo, l'impianto sperimentale, finora testato su dati simulati, potrebbe essere esteso all'analisi di serie storiche reali, con particolare riferimento ai mercati finanziari e alle variabili macroeconomiche. In tali contesti, spesso dominati da code pesanti, cluster di volatilità e cambiamenti strutturali, l'impiego di varianti robuste delle ELM, che sostituiscano la classica funzione di perdita quadratica con metriche maggiormente resilienti, potrebbe conferire ulteriore stabilità alle stime parametrizzate.

In secondo luogo, la spiccata trattabilità computazionale delle ELM suggerisce un loro naturale impiego all'interno di complesse procedure di ricampionamento basate sul *bootstrap*. Come accennato nei capitoli precedenti, replicare l'addestramento di una NNET standard per migliaia di campioni bootstrap risulta spesso proibitivo in termini di tempo; l'integrazione delle ELM consentirebbe, al contrario, di ottenere in tempi ridotti intere distribuzioni predittive, permettendo una quantificazione rigorosa e non parametrica dell'incertezza previsiva, nonché la costruzione di intervalli di previsione empirici e affidabili.

Infine, un ulteriore filone di sviluppo riguarda la struttura topologica delle reti NNET ed ELM. Cercando di indagare se il miglioramento delle performance computazionali riscontrato nel presente lavoro possa essere simile anche in presenza di architetture di reti NNET più articolate, con un numero maggiore di strati nascosti (reti deep) o con relative configurazioni di ELM pensate per questa tipologia di reti.

Bibliografia

[1] Cybenko (1989), Approximation by superpositions of a sigmoidal function; Mathematics of Control, Signals and Systems volume 2, pages 303-314

[2] A. Heinecke, J. Ho and W. Hwang (2020), Refinement and Universal Approximation via Sparsely Connected ReLU Convolution Nets; IEEE Signal Processing Letters, vol. 27, pp. 1175-1179

[3] Huang G.-B., Zhu Q.-Y., Siew C.-K. Extreme learning machine: a new learning scheme of feedforward neural networks 2004 IEEE International Joint Conference on Neural Networks, Vol. 2 (2004), pp. 985-990, 10.1109/IJCNN.2004.1380068

[4] Wang, Ran & Kwong, Sam & Wang, Xi-Zhao. (2012). A study on random weights between input and hidden layers in extreme learning machine. Soft Computing. 16. 10.1007/s00500-012-0829-1.

[5] Deng, W., Zheng, Q., & Chen, L. (2009). Regularized Extreme Learning Machine. In 2009 IEEE Symposium on Computational Intelligence and Data Mining (pp. 389–395). IEEE. DOI: 10.1109/CIDM.2009.4938676.

[6] Liu, X., Lin, S., Fang, J., Xu, Z., 2015. Is extreme learning machine feasible? A theoretical assessment (part I). IEEE Transactions on Neural Networks and Learning Systems 26, 7–20.

[7] Lin, S., Liu, X., Fang, J., Xu, Z., 2015. Is extreme learning machine feasible? A theoretical assessment (part II). IEEE Transactions on Neural Networks and Learning Systems 26, 21–34.

[8] La Rocca, M., Perna, C., 2019. Nonlinear autoregressive sieve bootstrap based on extreme learning machines. Mathematical Biosciences and Engineering 17, 636–653.

[9] La Rocca, M., Perna, C. (2023). Neural Network Bootstrap Forecast Distributions with Extreme Learning Machines. In: Iliadis, L., Maglogiannis, I., Alonso, S., Jayne, C., Pimenidis, E. (eds) Engineering Applications of Neural Networks. EANN 2023. Communications in Computer and Information Science, vol 1826. Springer, Cham. https://doi.org/10.1007/978-3-031-34204-2_43